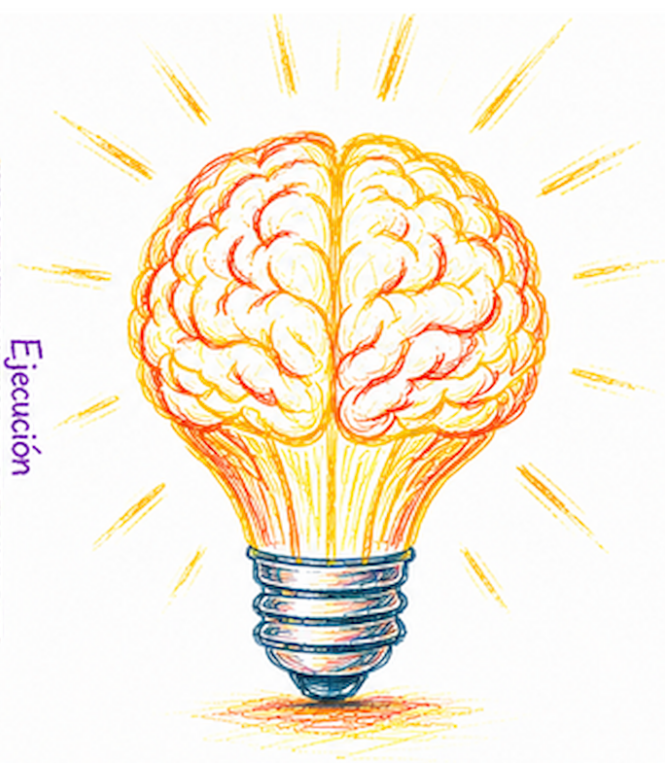
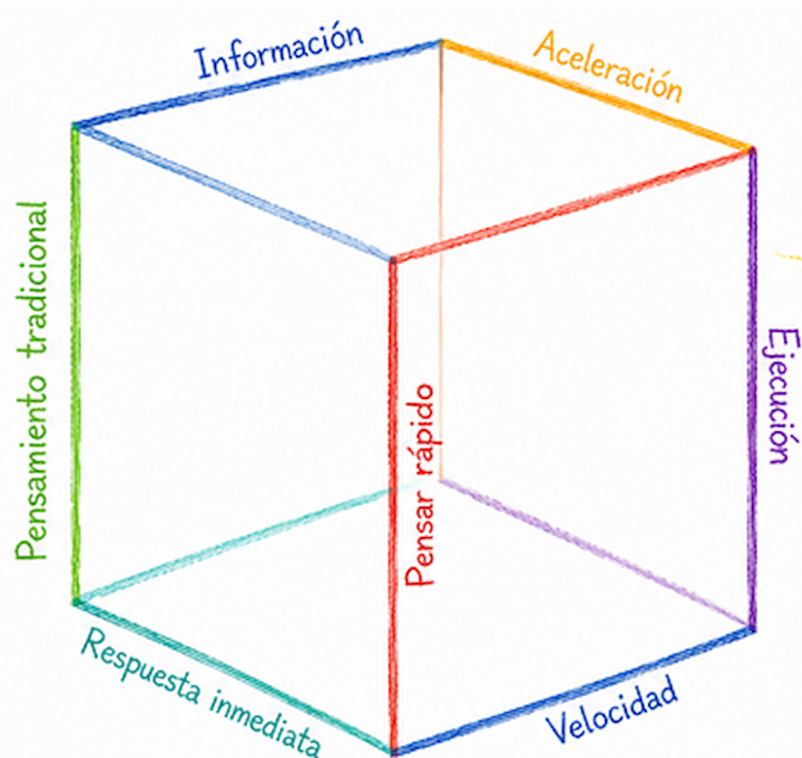


PENSANDO FUERA DE LA CAJA

IA y programación para
programadores y no programadores



Luis Alfonso Salazar Vaca

23 de agosto de 2026

Pensando fuera de la caja
IA y programación para programadores y no programadores

Luis Alfonso Salazar Vaca

23 de agosto de 2026

Contenidos

I	La caja	1
1	Cómo nos enseñaron a resolver	2
1.1	El reflejo de empezar rápido	2
1.2	Tres formas habituales de atacar un problema	2
1.2.1	Enumerar los casos	2
1.2.2	Reducir los casos aprovechando regularidades	3
1.2.3	Explorar las consecuencias	3
1.3	Lo que las tres tienen en común	3
1.4	Un problema que todos sabemos resolver	5
1.5	Por qué el tres en raya es el ejemplo perfecto	5
2	La solución que no convencía	7
2.1	El encargo: una máquina que no pueda perder	7
2.2	Primer intento: escribir todos los casos	7
2.3	Segundo intento: aprovechar la simetría	9
2.4	La desproporción que incomoda	10
2.5	Lo que buscaba no era rapidez, era entender	11
2.6	La decisión de parar	11
II	Buscando ver el bosque, un árbol a la vez	12
3	Detenerse antes de resolver	13
3.1	Soltar la papa caliente	13
3.2	Por qué los árboles no dejan ver el bosque	13
3.3	Dedicar tiempo a no resolver sino a pensar una mejor solución	14
3.4	Resolver no es optimizar	15
3.5	Un respiro en la era de las respuestas instantáneas	16
4	La pregunta que lo cambia todo	17
4.1	«¿Cómo resuelvo esto?» frente a «¿qué otra cosa tiene esta forma?»	17
4.2	Buscar por coincidencia de forma	17
4.3	El problema y su representación no son lo mismo	18
4.4	Qué significa que dos problemas sean equivalentes	19
4.5	El momento del hallazgo	20

III	Manos a la obra	22
5	El cuadrado mágico	23
5.1	Qué es y por qué aparece aquí	23
5.2	La constante mágica	23
5.3	Ocho y ocho: ¿casualidad?	24
5.4	La correspondencia exacta	24
5.5	Las simetrías de ambos objetos	26
5.5.1	Lo que la transformación conserva	27
6	Construyendo el algoritmo, etapa por etapa	28
6.1	Etapa 1: representar sin coordenadas	28
6.2	Etapa 2: saber quién ha ganado	30
6.3	Etapa 3: ver las amenazas	31
6.4	Etapa 4: decidir dónde jugar	32
6.5	Etapa 5: las cuatro reglas completas	33
6.6	Una partida entera, sin dibujar el tablero	34
7	Comprobarlo de verdad	35
7.1	Jugar unas partidas no demuestra nada	35
7.2	Recorrer el juego entero	35
7.3	La regla que estaba mal y cómo la descubrí	36
7.4	Cero derrotas: qué significa exactamente	37
7.5	Los errores que conviene provocar a propósito	37
8	¿Era necesario el cuadrado mágico?	39
8.1	La objeción del lector curioso	39
8.2	Tres versiones del mismo algoritmo	39
8.3	La respuesta: sí, se puede quitar	40
8.4	Y sin embargo, qué se pierde al quitarlo	40
8.5	Una transformación parcial no rinde	41
IV	Lo que realmente cambió	43
9	Menos código, menos reglas, más entendimiento	44
9.1	Las cifras, medidas y no estimadas	44
9.2	Cuatro reglas frente a cuatro mil quinientos casos	44
9.3	Defender un programa leyéndolo	45
9.4	Lo que la transformación no hizo	46
10	El modelo de pensamiento	48
10.1	Los siete movimientos	48
10.2	Observar: separar el problema de su descripción	49

10.3	Abstraer: quitar lo que no es esencial	49
10.4	Buscar: qué otra estructura tiene esta forma	50
10.5	Transformar: qué hay que conservar	51
10.6	Resolver: aprovechar lo que la nueva forma regala	51
10.7	Volver: traducir la solución	52
10.8	Evaluar: ¿mereció la pena el rodeo?	52
11	Cuándo esto no funciona	54
11.1	Problemas que no admiten transformación útil	54
11.2	Cuando encontrarla cuesta más que resolver	55
11.3	Cuando la dificultad solo se muda de sitio	55
11.4	El riesgo de ver transformaciones donde no las hay	56
11.5	Cómo saber que te has equivocado	57
V	Capacidades aumentadas con Inteligencia Artificial	58
12	La herramienta que amplifica	59
12.1	Dónde estamos hoy	59
12.2	Lo que la IA hace extraordinariamente bien	59
12.3	Lo que sigue siendo humano	60
12.4	Poner en marcha habilidades que no dominas	61
12.5	El riesgo: una herramienta que desalienta la pausa	62
13	Prompts que hacen pensar	64
13.1	No pidas la solución, pide acompañamiento	64
13.2	Anatomía de un buen prompt	64
13.3	Los prompts de este libro, explicados	65
13.3.1	El de la etapa 2	65
13.3.2	El de la comprobación	66
13.3.3	El de la búsqueda de estructuras	66
13.4	Cómo pedirle a la IA que te lleve la contraria	66
13.5	Verificar lo que te responde	67
VI	El balance	69
14	Análisis de resultados	70
14.1	Qué medí y cómo	70
14.2	Número de casos que cubre cada solución	70
14.3	Tamaño del código	72
14.4	Coste por jugada	73
14.5	Verificación: cero derrotas	74
14.6	La comparación justa: simetría en ambos lados	74

14.7	Qué demuestran estas cifras	75
14.8	Qué no demuestran	75
14.9	Cómo reproducir las mediciones	75
Apéndices		77
A	Glosario	77
	Términos del método	77
	Términos matemáticos	78
	Términos del juego	78
	Términos de verificación	79
	Términos de inteligencia artificial	79
B	Para seguir leyendo	80
B.1	Sobre el caso de este libro	80
B.2	Sobre cómo se resuelven problemas	80
B.3	Sobre el estado de la inteligencia artificial	81
B.4	Sobre apartarse del problema	81
B.5	Sobre reformular un problema	82
B.6	Sobre enseñar a pensar así	82
Referencias		83
Más allá de la caja		84
Cambiar la forma de representar para cambiar la forma de resolver		84
La IA como ampliación de lo humano		84
El valor de la pregunta		85
La próxima entrega: Space Invaders y la orientación a objetos		85

I

La caja

Capítulo 1

Cómo nos enseñaron a resolver

1.1 El reflejo de empezar rápido

Piensa en la última vez que te plantearon un problema.

Es probable que empezaras a resolverlo casi de inmediato. No porque fueras imprudente, sino porque es lo que hemos aprendido a hacer. En el colegio, el que terminaba primero levantaba la mano. En una entrevista de trabajo, el silencio largo se interpreta como no saber. En una reunión, quien propone algo en el primer minuto parece más competente que quien pide pensarlo.

Hemos aprendido que resolver es una carrera. Y como toda carrera, premia arrancar.

Detengámonos a pensar · Lo que este libro va a discutir

No que empezar rápido esté mal. A menudo es exactamente lo correcto: muchos problemas son lo que parecen y se resuelven por donde parece.

Lo que este libro discute es el **reflejo**: arrancar sin haber preguntado antes si el problema está bien planteado. Porque cuando el problema no está bien planteado, la velocidad no ayuda. Solo hace que llegues antes al sitio equivocado.

Hay una escena que se repite en cualquier oficio. Alguien lleva horas peleando con algo, se levanta a por un café, y en el camino de vuelta se le ocurre la solución. Todo el mundo la ha vivido y casi nadie saca la conclusión. Contamos esa escena como una anécdota simpática sobre la inspiración, cuando en realidad está diciendo algo bastante concreto: mientras estaba delante del problema, no lo veía.

Este libro trata de eso. De qué ocurre en ese rato en el que uno se aparta, y de cómo provocarlo a propósito en lugar de esperar a que llegue.

1.2 Tres formas habituales de atacar un problema

Antes de discutir el reflejo, conviene mirar qué hacemos cuando lo seguimos. Ante un problema de decisión —tenemos un estado y hay que elegir qué hacer— la práctica habitual recurre a tres vías.

1.2.1 Enumerar los casos

La más directa. Se identifican las situaciones posibles y se escribe qué hacer en cada una. Si pasa esto, haz aquello; si pasa lo otro, haz esto otro.

Es la vía que casi todo el mundo sigue la primera vez, y tiene una virtud enorme: siempre funciona. Si has cubierto todos los casos, el programa es correcto. La duda no está en si funciona, sino en cuántos casos hay.

1.2.2 Reducir los casos aprovechando regularidades

La segunda vía es más astuta. Si muchos casos son en el fondo el mismo caso visto de otra manera, no hace falta escribirlos todos: basta reconocer la familia y tratarla junta.

En un tablero, por ejemplo, girarlo noventa grados no cambia el juego. Las cuatro esquinas son la misma esquina cuatro veces. Reconocer eso reduce mucho el trabajo.

1.2.3 Explorar las consecuencias

La tercera vía no enumera situaciones sino futuros. Se prueban las jugadas posibles, se ven las respuestas del rival, se prueban las respuestas a esas respuestas, y se elige el camino que lleve al mejor final.

Es la familia de los algoritmos de búsqueda, y en los juegos por turnos su representante más conocido se llama Minimax (Russell & Norvig, 2021). Es una idea potente y general: sirve para el tres en raya, para las damas y, con muchos añadidos, para el ajedrez.



Sin programar · Las tres vías, sin tecnicismos

Imagina que tienes que enseñarle a alguien a jugar.

Enumerar es darle una lista: «si ves esto, juega aquí».

Reducir es darle la misma lista, más corta, avisándole de que algunas situaciones son la misma girada.

Explorar es enseñarle a imaginar: «prueba una jugada, imagina qué haría el otro, y decide».

1.3 Lo que las tres tienen en común

Las tres vías funcionan. Las tres son legítimas. Y las tres comparten algo que no solemos mirar, porque está tan al principio que ni parece una decisión.

 Dónde empieza cada vía**Pensamiento tradicional**

Las tres arrancan igual: aceptan el problema **tal como viene descrito**.

Un tablero de nueve casillas, con filas, columnas y diagonales. Nadie pregunta si esa descripción es la más conveniente. Es *el problema*, y punto.

A partir de ahí, todo el esfuerzo se dedica a recorrerlo mejor.

Transformación estructural

Hay un paso anterior que nadie dio: preguntarse si el problema podría describirse de otra manera.

No resolverlo de otra manera. **Describirlo** de otra manera.

Y comprobar si, descrito de otro modo, hace falta menos esfuerzo para resolverlo.

Esa es la caja del título. No es una forma concreta de resolver: es la *descripción heredada* del problema, que aceptamos sin examinar porque llegó junto con el enunciado.

 Detengámonos a pensar · Esto tiene nombre, y está estudiado

No es una metáfora mía. La psicología cognitiva lleva décadas estudiándolo bajo tres nombres que una revisión reciente reúne en uno solo, *fijación*: el efecto *Einstellung*, el conjunto mental y la fijación funcional (Wang et al., 2023).

La descripción del fenómeno es exactamente la del párrafo anterior: la primera solución que viene a la cabeza **bloquea la búsqueda de otras mejores**, y lo hace sin que uno se dé cuenta.

Esa misma revisión, sobre cincuenta y tres estudios, encuentra dos cosas que interesan aquí. Entre las estrategias que sí funcionan para problemas cerrados —como el tres en raya— figuran el **cambio de representación** y la **incubación**, que son precisamente las dos que recorren este libro.

Con una salvedad que conviene no saltarse: eso vale para problemas *cerrados*. Para los abiertos, la misma revisión advierte que quitar la fijación no mejora automáticamente el resultado.

 Detengámonos a pensar · Y no basta con avisar de que existe

Un estudio de 2023 lo puso a prueba precisamente con programadores. Les dieron tareas donde había una vía familiar peor y otra mejor menos habitual, y probaron dos maneras de romper la fijación: cambiar los colores del editor, y cambiarlos además de pedirles expresamente que ignorasen sus soluciones anteriores.

Ninguna de las dos funcionó. Los autores concluyen que las técnicas probadas resultaron **insuficientes** para superar el conjunto mental (Sergeyuk et al., 2023).

Es un resultado negativo, y por eso lo traigo. Saber que la fijación existe no la desactiva. Que alguien te diga «piensa fuera de la caja» tampoco. Hace falta algo más concreto, y de eso trata el resto del libro.

💡 Detengámonos a pensar · El corazón del libro

Todo lo que viene después —un juego, un cuadrado mágico, código, mediciones— está al servicio de una sola idea:

antes de buscar cómo resolver un problema, pregúntate si lo estás mirando desde el sitio correcto.

El algoritmo que vamos a construir es el vehículo. Ese cambio en la forma de pensar es el destino.

1.4 Un problema que todos sabemos resolver

Para discutir esto hace falta un problema. Y el problema tiene que cumplir una condición incómoda: debe ser lo bastante sencillo como para que puedas comprobar todo lo que este libro afirme, sin fiarte de nadie.

Vamos a usar el tres en raya.

Sí, ese juego. El de la cuadrícula de nueve casillas, las cruces y los círculos, el que se juega en los márgenes de los cuadernos y que todo el mundo abandona hacia los diez años, cuando descubre que si los dos jugadores saben lo que hacen siempre acaba en tablas.

🕒 Inténtalo primero · Antes de seguir

Coge papel. Dibuja el tablero.

Escribe, en lenguaje normal, las instrucciones que le darías a alguien para que jugara bien. No a un ordenador: a una persona que no conoce el juego.

Hazlo de verdad, no lo imagines. Tardarás cinco minutos y esas instrucciones te van a servir durante todo el libro.

Guárdalas. Volveremos a ellas más de una vez.

1.5 Por qué el tres en raya es el ejemplo perfecto

La elección puede parecer decepcionante. Un libro sobre cambiar la forma de pensar, ¿y el ejemplo es un juego de niños?

Precisamente por eso.

Porque lo conoces. Ninguna dificultad del libro vendrá del enunciado. Todo lo que te cueste será lo que de verdad quiero que te cueste.

Porque lo puedes recorrer completo. El juego entero, con todas sus partidas posibles, se explora en un ordenador cualquiera en menos de un segundo. Eso significa que cada afirmación de este libro se puede *comprobar*, no creer. Es una propiedad rarísima: en casi ningún problema real puedes examinar todos los casos.

Porque ya está resuelto. No hay ninguna gloria en resolver el tres en raya, y eso libera. No vamos a competir con nadie. Vamos a mirar cómo se resuelve, que es distinto.

Y porque, siendo tan pequeño, resiste. Esto es lo que menos se espera. Verás en el capítulo siguiente que la vía más natural para programarlo produce miles de casos. Miles,

para nueve casillas.

Capítulo 2

La solución que no convencía

2.1 El encargo: una máquina que no pueda perder

El problema que originó todo esto llegó con una condición muy concreta:

La computadora **no puede perder**. Solo puede empatar o ganar.

Merece la pena detenerse en cómo está formulada, porque la formulación ya contiene una elegancia que se suele pasar por alto.

No dice «que juegue bien». No dice «que gane lo más posible». Dice que no pierda. Y en el tres en raya perder solo es posible si se comete un error, de modo que la condición equivale a exigir un juego *sin errores*.

💡 Detengámonos a pensar · Una condición que se puede comprobar

«Jugar bien» es una aspiración: no se sabe cuándo se ha cumplido.

«No perder nunca» es una propiedad: o se cumple o no, y se puede verificar.

Cuando alguien te plantee un problema, fíjate en si el criterio de éxito es comprobable. Si no lo es, la primera transformación útil suele ser convertirlo en uno que lo sea.

Había además una segunda condición, esta vez involuntaria y menos habitual: cuando resolví este problema **no tenía conocimiento sobre Minimax**. No lo descarté por sus limitaciones. Sencillamente no sabía que existía.

Conviene tenerlo presente, porque explica el camino que sigue. No rechacé la herramienta estándar: tuve que resolver sin ella.

2.2 Primer intento: escribir todos los casos

La vía natural, cuando no se conoce nada mejor, es enumerar.

Se mira el tablero, se identifican las situaciones y se escribe qué hacer en cada una. Si el rival tiene dos en la fila de arriba y la tercera está libre, bloquea ahí. Si yo tengo dos en la diagonal, cierro. Si el centro está libre, cógelo.

Empiezas escribiendo cinco o seis condiciones y parece que va bien. Después aparecen las excepciones. Después las excepciones de las excepciones.

🔗 Inténtalo primero · Antes de mirar la cifra

Nueve casillas. Dos jugadores. Un juego que aprendiste de niño.

¿Cuántas situaciones distintas crees que tendría que cubrir un programa que enumerase todos los casos?

Di un número antes de seguir leyendo. En voz alta, o escríbelo. Solo así funciona.

Las cifras que siguen no son estimaciones. Están contadas sobre el juego, una por una, y además se pueden calcular, que es mejor que creerlas.

Contadas así, son 4 520 las situaciones distintas ante las que un programa tiene que decidir una jugada. Y como las posiciones que aparecen si abres tú no son las que aparecen si abre el rival, un programa que juegue en los dos papeles se enfrenta a 8 533 posiciones distintas.

Cuadro 2.1: Tamaño real del problema. Todas las cifras están contadas o calculadas sobre el juego; la cadena que lleva de una a la siguiente está en el capítulo 14.

Magnitud	Valor
Combinaciones de las nueve casillas	19 683
Posiciones alcanzables	5 478
Ya decididas, sin jugada que elegir	958
Hay que decidir jugada, empezando	2 423
Hay que decidir jugada, respondiendo	2 097
Casos que cubrir	4 520
Posiciones legales alcanzables, en ambos papeles	8 533

Cuatro mil quinientas veinte situaciones. Para un tablero de nueve casillas.

✓ Compruébalo · Si quieres ver de dónde sale cada cifra

Ninguna de estas cifras hay que creérsela, y este libro no te pide que lo hagas. Cada una se calcula a partir de la anterior, salvo unas pocas que exigen recorrer el juego en lugar de deducirlo de una fórmula.

La cadena completa —la fórmula, la suma término a término y qué cifras hay que ejecutar— está en el capítulo 14, donde el libro reúne el recuento del juego. Va allí y no aquí a propósito: en este punto del relato lo que importa es la desproporción, no la aritmética que la establece.

Si se escribe una condición por caso, el número de condicionales no hay que medirlo: es el propio recuento de casos, uno por cada uno.

Cuadro 2.2: Implementación por enumeración de casos. Los condicionales se siguen del recuento; las líneas y los bytes están medidos sobre el archivo generado.

Medida	Cómo se obtiene	Valor
Condicionales	un caso, un condicional: $2\,423 + 2\,097$	4 520
Líneas de código	medido	4 522
Tamaño del archivo	medido	239 435 bytes

💡 **Detengámonos a pensar · La pregunta que nadie hace**

Doscientos treinta y nueve kilobytes de programa para un juego que cabe en una servilleta. La reacción normal ante esto es buscar una técnica mejor. La reacción que propone este libro es otra, y es una pregunta:

¿de dónde han salido esos cuatro mil quinientos casos?

No estaban en el juego. El juego tiene nueve casillas y ocho formas de ganar. Los cuatro mil quinientos casos aparecieron cuando decidimos describir el juego de una manera determinada.

Los pusimos nosotros.

2.3 Segundo intento: aprovechar la simetría

Existe una mejora conocida, y es buena.

Muchas de esas posiciones son la misma vista de otro modo. Un tablero girado noventa grados es el mismo tablero. Reflejado en un espejo, también. Un tablero de tres por tres admite ocho transformaciones de ese tipo: cuatro giros y cuatro reflexiones.

Si el programa reconoce que dos posiciones son equivalentes, no necesita tratarlas por separado.

Cuadro 2.3: Reducción obtenida al aprovechar las simetrías. Los casos con simetría son las clases de equivalencia de los 4 520 bajo las ocho transformaciones del tablero: 338 empezando y 289 respondiendo. Los condicionales van uno por clase; las líneas y los bytes están medidos.

Medida	Sin simetría	Con simetría	
Casos que cubrir	4 520	627	338 + 289
Condicionales	4 520	627	uno por clase
Líneas de código	4 522	630	medido
Tamaño	239 435	35 215	medido

La mejora es real y considerable: siete veces menos. Nadie discute que sea un avance.

Fíjate en que no son ocho veces menos, que es lo que cabría esperar de ocho transformaciones. La división limpia daría $4\,520/8 = 565$ y la cifra real es 627: la reducción es de 7,21 veces, no de 8.

La diferencia no es un error de cuenta, y explica lo que una simetría hace de verdad. Hay tableros que **quedan iguales a sí mismos** al girarlos o reflejarlos —el tablero vacío, por ejemplo, o el que solo tiene el centro ocupado—. Esos no tienen ocho versiones distintas, sino menos, y cada uno forma una clase más pequeña. Cuantas más clases pequeñas, más clases en total.

✓ Compruébalo · De dónde sale el 627

Cada uno de los 4 520 casos pertenece a una clase, y la clase tiene tantos miembros como versiones distintas produzcan las ocho transformaciones.

Versiones distintas	Casos	Clases
8	4 064	508
4	444	111
2	8	4
1	4	4
total	4 520	627

Las dos columnas se comprueban una contra otra: $508 \times 8 + 111 \times 4 + 4 \times 2 + 4 \times 1 = 4\,520$, y las clases suman 627. Los cuatro casos con una sola versión son los tableros que cualquier giro y cualquier reflexión dejan intactos.

✋ Sin programar · Qué acaba de pasar, en una frase

Hemos pasado de una lista de cuatro mil quinientas instrucciones a una lista de seiscientas veintisiete.

Sigue siendo una lista.

2.4 La desproporción que incomoda

Aquí es donde este libro empieza de verdad.

Seiscientos veintisiete casos siguen siendo demasiados. No demasiados para el ordenador —le da igual—, sino demasiados **para una persona**.

🔗 Qué sabemos del programa que acabamos de escribir

Lo que sabemos

Funciona. Pasa todas las pruebas. Nunca pierde.

¿Por qué nunca pierde? Porque los seiscientos veintisiete casos están bien.

¿Y cómo lo sabemos? Porque los hemos probado.

Nadie ha leído los seiscientos veintisiete casos y ha entendido por qué el conjunto es correcto. Nadie puede.

Lo que eso significa

Un programa cuya corrección reside en una lista es un programa en el que hay que *confiar*.

Puede explicarse por qué hizo *esta* jugada: se busca el caso y se lee.

Pero no puede explicarse por qué juega bien *siempre*. Esa respuesta está repartida entre seiscientos veintisiete sitios, y nadie la tiene entera.

Esa es la desproporción: un juego que un niño aprende en dos minutos produce un programa que ningún adulto puede leer y defender.

Y la desproporción no es un problema técnico. Es una **señal**.

 Detengámonos a pensar · Aprende a reconocer esta señal

Cuando la descripción de un problema es mucho más complicada que el problema mismo, casi siempre ocurre una de dos cosas:

- el problema es genuinamente complicado y no hay nada que hacer, o
- la complicación la ha introducido la descripción.

Distinguir entre las dos es difícil, y no hay método infalible. Pero hay algo que se puede hacer siempre: **sospechar**.

La mayoría de la gente no sospecha, porque el programa funcionaba.

2.5 Lo que buscaba no era rapidez, era entender

Llegamos al punto que da sentido a todo lo demás, y conviene enunciarlo con mis propias palabras:

Era indispensable encontrar una solución más creativa al problema, sobre la cual pudiera tener completo entendimiento de su funcionamiento, para garantizar las afirmaciones que de ella deriven.

Léelo otra vez. No dice «más rápida». No dice «más corta». Dice **sobre la cual pudiera tener completo entendimiento**.

Y da la razón, que es lo importante: para poder *garantizar las afirmaciones que de ella deriven*. Es decir, para poder responder por lo que el programa hace.

 Detengámonos a pensar · El criterio que va a guiar todo el libro

Casi toda la ingeniería mide el rendimiento: cuánto tarda, cuánta memoria gasta, cuántas operaciones hace.

Aquí el criterio es otro: **¿puedo entenderlo del todo?**

No es un criterio blando. Es más exigente que el rendimiento, porque el rendimiento se mide con un cronómetro y esto se mide leyendo, que es más difícil de aprobar.

Y tiene una consecuencia práctica inmediata: descarta las dos soluciones que teníamos. Ninguna de las dos se puede entender del todo. Funcionan, pero no se pueden defender.

2.6 La decisión de parar

Con las dos vías descartadas por un criterio que ellas no podían cumplir, quedaba una situación incómoda: no había solución a la vista, y las conocidas no servían.

Lo que decidí hacer entonces es el asunto del capítulo siguiente, y es lo menos parecido a resolver que puede uno imaginarse.

Dejé de intentar resolverlo y me concentré en otras actividades.

II

Buscando ver el bosque, un árbol a la vez

Capítulo 3

Detenerse antes de resolver

Dejamos el capítulo anterior en un punto incómodo. Las dos vías conocidas estaban descartadas, no porque fallaran —funcionaban— sino porque ninguna podía entenderse del todo. Y no había una tercera a la vista.

Lo que hice entonces es lo más parecido a rendirse que puede uno imaginar, y es el asunto de este capítulo.

3.1 Soltar la papa caliente

Lo llamo así, y la expresión es mejor que cualquier tecnicismo: *soltar la papa caliente*.

Consiste en apartarse del problema. Levantarse. Dejar de mirarlo. No como descanso ni como abandono, sino como parte del trabajo.

Muchas de las veces nos estorban los árboles para poder observar el bosque.

Es necesario alejarse un poco de la situación, enfriar la cabeza, para poder concentrarnos de manera efectiva y hallar la solución adecuada.

Fíjate en el orden de la frase, porque es contraintuitivo. Alejarse no viene *después* de concentrarse: viene **antes**. Es la condición para poder concentrarse de verdad.

Inténtalo primero · Comprueba tu propia experiencia

Antes de seguir, recuerda una vez en que se te ocurriera la solución a algo fuera del sitio donde lo estabas peleando: en la ducha, andando, a punto de dormirte, fregando los platos.

Casi todo el mundo tiene ese recuerdo. Ahora la pregunta:

¿Qué hiciste con esa observación? ¿La incorporaste a tu forma de trabajar, o la archivaste como una anécdota curiosa sobre la inspiración?

3.2 Por qué los árboles no dejan ver el bosque

La metáfora es tan común que ha perdido filo, así que conviene devolvérselo mirando qué dice exactamente.

Estar cerca de un problema tiene una ventaja obvia: se ven los detalles. Y un coste que no es obvio: **solo** se ven los detalles.

Cuando llevas dos horas escribiendo condiciones para el tres en raya, tu atención está ocupada en si falta cubrir el caso de la diagonal cuando el rival tiene el centro. Esa es una pregunta legítima y hay que responderla. Pero mientras la respondes, hay una pregunta que no te puedes hacer, porque no cabe: *¿por qué estoy escribiendo condiciones?*

🔍 Dos distancias, dos preguntas

Cerca del problema

Cerca. Se ven los detalles. Las preguntas son de la forma «¿cubrí este caso?», «¿está bien esta comparación?», «¿falta algo aquí?».

Son preguntas *dentro* del planteamiento. Ninguna de ellas puede poner en duda el planteamiento, porque todas lo presuponen.

Lejos del problema

Lejos. Se pierden los detalles y aparece la forma. Las preguntas cambian: «¿por qué hay tantos casos?», «¿de dónde salen?», «¿y si esto no fuera un problema de casos?».

Son preguntas *sobre* el planteamiento. Solo caben cuando el detalle ha dejado de ocupar sitio.

Ahí está el mecanismo, y no tiene nada de místico. No es que la distancia traiga ideas. Es que **libera la atención que el detalle tenía secuestrada**, y solo entonces caben las preguntas grandes.

💡 Detengámonos a pensar · Un aviso de honestidad

Este libro va a apoyarse mucho en la pausa, así que conviene decir con claridad qué se sabe y qué no.

Que apartarse del problema ayuda es una observación antigua y compartida por mucha gente que trabaja creando cosas. Es también un fenómeno estudiado formalmente —se lo llama *incubación*— y sobre el que existe literatura científica.

Ahora bien: **la evidencia es discutida**. Las revisiones recientes señalan que las definiciones no están estandarizadas, que los diseños experimentales varían mucho y que los resultados no son concluyentes (Kuliev, 2024).

Es decir: lo que aquí se cuenta es lo que me funcionó a mí, y encaja con algo que la ciencia estudia sin haberlo cerrado. No es una técnica demostrada. Tómallo como lo que es.

3.3 Dedicar tiempo a no resolver sino a pensar una mejor solución

Hay un detalle en el relato original que es fácil pasar por alto y que cambia el sentido de todo.

La pausa no fue un accidente. No ocurrió que la solución llegara sola mientras yo hacía otra cosa. Dedicué **varias horas deliberadamente** a buscar otra vía.

💡 Detengámonos a pensar · Incubación no es distracción

La diferencia entre las dos es enorme y casi nadie la señala.

Distraerse es dejar de pensar en el problema.

Incubar es dejar de pensar en *la solución que estabas intentando*, sin dejar de tener el problema encima.

En la segunda, sigues trabajando. Lo que has soltado es el intento, no el encargo.

Esto tiene una consecuencia práctica incómoda: el tiempo de la pausa es **tiempo de trabajo**, y hay que reclamarlo como tal.

Es difícil. En casi ningún sitio se puede decir «voy a pasar la tarde sin escribir código porque estoy buscando otra forma de plantear esto». Se interpreta como no avanzar. Y sin embargo, en este caso, esas horas produjeron la diferencia entre un programa de doscientos treinta y nueve kilobytes y uno que se puede leer de una sentada.

Sin programar · La versión sin ordenadores

Esto no va de programar. Ocurre igual escribiendo un informe que no arranca, preparando una clase que no se sostiene o discutiendo un presupuesto que no cuadra.

La escena es siempre la misma: llevas rato dándole vueltas a *cómo* hacerlo mejor, cuando la pregunta que falta es si hay que hacerlo así.

3.4 Resolver no es optimizar

Añado una condición que conviene no perder, porque desactiva una objeción que llegará sola:

Esto no implica que la solución sea perfecta o esté optimizada. Seguramente requerirá trabajo a futuro para mejorarla. Pero debe servir para resolver el problema planteado.

Dos actividades que se confunden constantemente:

Cuadro 3.1: Dos actividades distintas, con criterios distintos.

	Resolver	Optimizar
Pregunta	¿Funciona?	¿Puede funcionar mejor?
Criterio	Cumple lo pedido	Cumple mejor algún indicador
Cuándo acaba	Cuando resuelve	Nunca del todo
Qué se cambia	El planteamiento	Los detalles

Detengámonos a pensar · Por qué esto importa aquí

Cambiar la forma de plantear un problema pertenece a la primera columna.

Si te exiges que la nueva forma sea además más rápida que la anterior desde el primer momento, no la vas a encontrar nunca: las ideas nuevas nacen sin pulir, y compiten contra ideas viejas que llevan años puliéndose.

La pregunta correcta ante una solución recién nacida no es «¿es la mejor?», sino «¿resuelve, y la entiendo?». Lo demás viene después, si viene.

3.5 Un respiro en la era de las respuestas instantáneas

Todo lo anterior era ya difícil hace veinte años. Hoy lo es más, y conviene mirarlo de frente porque este libro te va a pedir precisamente eso.

Tienes al alcance una herramienta que responde en segundos. Escribes «hazme un tres en raya que no pierda nunca» y en menos de un minuto tienes código que funciona. Probablemente use Minimax, probablemente esté bien, y probablemente no sepas decir por qué funciona.

Qué se gana y qué se pierde

Lo que se gana

Se gana: tiempo, mucho. Un problema resuelto en un minuto en lugar de en una tarde. Acceso a técnicas que no conocías. La posibilidad de construir cosas que no sabrías construir solo. Nada de esto es poco. Sería absurdo despreciarlo.

Lo que se pierde

Se pierde: el rato de desconcierto. Ese rato en el que no sabes cómo seguir es exactamente donde ocurre lo que este libro quiere enseñarte. Si lo eliminas, obtienes la solución y te quedas sin la pregunta que la habría hecho tuya.

Detengámonos a pensar · El riesgo, dicho sin rodeos

Una herramienta que responde al instante **desalienta la pausa**.

Si el reflejo ante cualquier problema pasa a ser preguntar antes de pensar, habremos multiplicado la velocidad para resolver y atrofiado la capacidad de replantear.

Detenerse antes de resolver es hoy *más* necesario que antes, no menos. Precisamente porque nunca fue tan fácil no detenerse.

Esto no es un argumento contra usar la inteligencia artificial. Este libro te va a pedir que la uses, y trae los prompts para hacerlo. Es un argumento sobre *cuándo*: primero la pregunta, después la herramienta.

En el capítulo 12 volveremos sobre ello con calma. De momento basta con una regla de bolsillo: cuando notes que vas a pedirle algo a una máquina, comprueba antes que sabes qué le estás pidiendo y por qué eso, y no otra cosa.

Queda dicho, entonces, que apartarse del problema es trabajo y no abandono. Lo que no dice este capítulo es qué se hace con la cabeza despejada, porque apartarse por sí solo no resuelve nada: se puede volver descansado y con la misma pregunta de antes. Lo que cambió las cosas fue *otra* pregunta, y de ella se ocupa el capítulo siguiente.

Capítulo 4

La pregunta que lo cambia todo

La pausa, por sí sola, no resuelve nada. Puedes apartarte del problema una tarde entera y volver exactamente igual.

Lo que hace que la pausa sirva es qué preguntas puedes hacerte durante ella. Y hay una, en concreto, que cambió este problema.

4.1 «¿Cómo resuelvo esto?» frente a «¿qué otra cosa tiene esta forma?»

Cuando volví al problema después de apartarme, no volví con una idea. Volví con otra pregunta.

Las dos preguntas

La pregunta natural

¿Cómo resuelvo esto?

Es la pregunta natural, y es la que casi todos hacemos. Da por bueno el problema tal como está descrito y pide un camino desde ahí.

Sus respuestas son técnicas: enumerar, buscar, optimizar, aprender. Todas mejoran el recorrido. Ninguna cuestiona el terreno.

Te **encierra** en el problema.

La pregunta de este libro

¿Qué otra cosa tiene esta misma forma?

No pide una solución. Pide *otro problema*. Sus respuestas no son técnicas: son objetos. Una estructura matemática, un juego distinto, un fenómeno de otro campo que resulta tener la misma forma.

Te **obliga a salir** para poder mirarlo desde fuera.

Detengámonos a pensar · Léelo otra vez, porque es todo el libro

La segunda pregunta **no busca una solución. Busca otro problema.**

Es tan contraintuitivo que cuesta sostenerlo. Estás atascado, y en lugar de buscar la salida te pones a buscar otro laberinto.

La apuesta es que si encuentras uno con la misma forma y que ya está resuelto, la salida te la regalan.

4.2 Buscar por coincidencia de forma

Aquí viene el detalle que convierte esto en un método y no en una ocurrencia. Lo dije así:

Buscaba un problema que coincidiera matricialmente con las dimensiones del tres en raya que estaba buscando resolver.

Mi criterio de búsqueda fue la **coincidencia de forma**. Concretamente, de dimensión: una estructura de tres por tres.

No busqué «algo relacionado con juegos». Ni «algo que sirva para decidir». Busqué algo con la misma forma, aunque no tuviera nada que ver con el asunto.

 Inténtalo primero · Practica el movimiento antes de ver la respuesta

Olvida por un momento que sabes cómo acaba esto.

Tienes delante una rejilla de tres por tres. Nueve posiciones. Ocho maneras de recorrerla en línea.

Haz una lista de todo lo que se te ocurra que tenga esa forma. No filtres, no descartes por absurdo, no busques que sea útil. Solo forma.

Apunta al menos cinco cosas. Después sigue leyendo.

 Sin programar · Cómo se busca por forma

Buscar por forma es un músculo, y se entrena describiendo el problema *sin nombrar de qué va*.

En lugar de «tres en raya», di: «nueve posiciones organizadas en tres filas de tres, con ocho conjuntos de tres posiciones que cuentan como especiales».

Ahora la pregunta «¿qué más tiene esta forma?» se puede responder, porque ya no estás pensando en un juego. Estás pensando en una forma.

4.3 El problema y su representación no son lo mismo

Para que la pregunta funcione hay que aceptar antes una distinción que parece obvia enunciada y que casi nunca se aplica.

Definición 4.1 (Problema y representación). El *problema* es lo que hay que conseguir: los estados posibles, las reglas que los relacionan y la condición que decide cuándo se ha logrado.

La *representación* es la forma concreta en que escribimos todo eso para poder manipularlo.

Un mismo problema admite muchas representaciones. Todas describen lo mismo. No todas hacen visible lo mismo.

 Detengámonos a pensar · Un ejemplo que ya conoces

Multiplica **XLVII** por **XIX**.

Ahora multiplica **47** por **19**.

Es la misma operación, con los mismos números y el mismo resultado. El problema no ha cambiado en absoluto.

Lo que ha cambiado es que en la segunda representación existe un procedimiento que puedes ejecutar, y en la primera no. Durante siglos, media Europa multiplicó mal por

culpa de la representación, no por culpa de la aritmética.

Nadie resolvió ese problema encontrando un algoritmo mejor para los números romanos.

La distinción tiene una consecuencia práctica que casi nunca se explota: cuando un problema resulta difícil, la dificultad puede estar en el problema o puede estar en la representación. **Y son cosas distintas, con remedios distintos.**

Si está en el problema, hay que trabajar más. Si está en la representación, trabajar más es exactamente lo que no ayuda.

De dónde venían los 4 520 casos

Parecen parte del juego

Se dan por parte del problema. «El tres en raya tiene miles de situaciones» suena a hecho sobre el juego.

Y con esa creencia, la única salida razonable es buscar una técnica que gestione miles de situaciones.

Los pusimos al describirlo

Ahora la distinción de esta sección lo explica. Los casos no pertenecían al problema: pertenecían a *una* de sus representaciones, la cuadrícula que hay que inspeccionar casilla por casilla.

Cámbiala y desaparecen, sin haber tocado el juego. Es la prueba de que estaban en la descripción y no en lo descrito.

4.4 Qué significa que dos problemas sean equivalentes

Si vamos a cambiar de representación, hace falta un criterio. De lo contrario cambiaríamos el problema en lugar de cambiar cómo lo miramos, y la solución que encontremos no serviría para nada.

Dos representaciones sirven para lo mismo cuando se cumplen tres condiciones.

Primera: correspondencia de estados. Todo estado de una tiene su estado en la otra, y al revés. Nada se pierde y nada se inventa.

Segunda: conservación de lo relevante. Lo que decide si el problema está resuelto debe seguir decidiéndolo después de traducir. Si en el original se gana con tres en línea, en la nueva representación tiene que haber algo que signifique exactamente eso: ni de más, ni de menos.

Tercera: vuelta. La solución hallada en la nueva representación tiene que poder traducirse de vuelta. Si te quedas con una respuesta preciosa que no sabes qué significa en el problema original, no has resuelto nada.

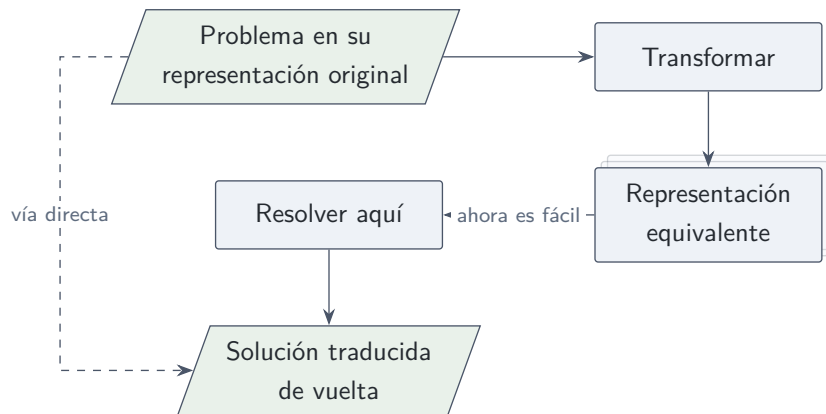


Figura 4.1: El rodeo. Solo merece la pena si cuesta menos que el camino directo y conserva lo que hace válida la solución.

💡 Detengámonos a pensar · La condición que gobierna todo

El esquema contiene ya la trampa en la que es fácil caer:

el rodeo solo tiene sentido si cuesta menos que el camino directo.

Encontrar la transformación cuesta tiempo. Traducir cuesta esfuerzo. Volver, también.

Si el problema se resolvía en diez minutos por la vía directa, todo esto es un lujo.

Saber cuándo compensa y cuándo no es lo que separa un método de una manía. El capítulo 11 se ocupa de ello sin adornos.

4.5 El momento del hallazgo

Volvamos a la rejilla de tres por tres y a la lista que te pedí que hicieras.

Lo que encontré fue el **cuadrado mágico de orden tres**: una disposición de los números del uno al nueve en la que cada fila, cada columna y cada diagonal suman lo mismo.

Nada que ver con el tres en raya. Ni juego, ni jugadores, ni turnos. Solo una rejilla de tres por tres con una propiedad aritmética.

Coincidió la forma. Eso era todo lo que buscaba.

💡 Detengámonos a pensar · Lo que ocurrió después no estaba previsto

Y aquí está lo que hace este caso digno de un libro.

Busqué por *coincidencia de dimensión*: una estructura de tres por tres. Es lo único que exigía el criterio.

Lo que encontré resultó tener, además, una coincidencia mucho más profunda que la buscada. No solo la misma forma: la misma *estructura interna*.

Ese regalo no formaba parte del plan. Y es honesto decirlo, porque si el libro lo presentara como consecuencia del método estaría vendiéndote algo que el método no garantiza.

Lo que el método garantiza es que buscas. Que encuentres, y cuánto, depende del problema.

El segundo paso cabe en una frase: *«pensé en cómo este problema se podría adaptar a mi*

situación».

Merece la pena subrayarlo porque desmonta la imagen romántica del hallazgo. La estructura no la apliqué tal cual, como si encajara sola igual que una pieza de puzle. Hubo que trabajarla. El hallazgo fue el principio del trabajo, no el final.

🔗 Inténtalo primero · Antes de pasar al taller

Sabes que el tablero tiene ocho maneras de ganar.

Ahora te digo que el cuadrado mágico de orden tres tiene ocho líneas que suman lo mismo.

¿Qué harías tú con esa información? ¿Se te ocurre cómo podría un juego de posiciones convertirse en un problema de sumas?

No hace falta que lo resuelvas. Basta con que lo intentes cinco minutos. El capítulo siguiente será distinto según lo hayas intentado o no.

Con esto termina la parte de las preguntas. La forma ya está encontrada, y sabemos qué habría que exigirle a la correspondencia para poder fiarnos de ella. Lo que no sabemos todavía es si la cumple.

Eso se demuestra, no se supone, y es lo primero que hace la parte siguiente.

III

Manos a la obra

Capítulo 5

El cuadrado mágico

Ocho maneras de ganar. Ocho líneas que suman lo mismo.

Si has hecho el último reto del capítulo anterior, llevas cinco minutos dándole vueltas a esa coincidencia. Si no lo has hecho, todavía estás a tiempo: lo que viene se lee de otra manera según lo hayas intentado o no.

A partir de aquí dejamos de hablar de cómo se piensa y empezamos a construir.

5.1 Qué es y por qué aparece aquí

Recordemos dónde estábamos. Hay un tablero de nueve casillas y ocho formas de ganar. Y hay una insatisfacción: todas las vías conocidas exigían describir el problema con más detalle del que el problema parecía tener.

 **Detengámonos a pensar · Antes de buscar una solución, buscar otro problema**

El movimiento que sigue no es el habitual. En lugar de preguntarme «¿cómo resuelvo esto?», la pregunta que me hice fue «¿qué otra cosa tiene esta misma forma?».

Es un cambio pequeño en el enunciado y enorme en las consecuencias. La primera pregunta te encierra en el problema. La segunda te obliga a salir de él para mirarlo desde fuera.

El tablero es una matriz de tres por tres. La búsqueda, entonces, fue de un objeto matemático con esa misma dimensión. Y lo hay.

Definición 5.1 (Cuadrado mágico). Un *cuadrado mágico* de orden n es una matriz $M \in \mathbb{N}^{n \times n}$ que contiene cada uno de los enteros de 1 a n^2 exactamente una vez, y en la que la suma de los elementos de cada fila, de cada columna y de cada una de las dos diagonales principales toma un mismo valor, llamado *constante mágica*.

Para $n = 3$ existe uno solo, salvo simetrías. Lo llamaremos *matriz de referencia*:

$$M = \begin{pmatrix} 2 & 7 & 6 \\ 9 & 5 & 1 \\ 4 & 3 & 8 \end{pmatrix} \quad (5.1)$$

Cada fila, cada columna y cada diagonal suman quince.

5.2 La constante mágica

La constante no hay que memorizarla; se deduce. La suma de todos los elementos de un cuadrado mágico de orden n es $\frac{n^2(n^2+1)}{2}$, y las n filas suman todas lo mismo y cubren la

matriz entera:

$$K_n = \frac{1}{n} \cdot \frac{n^2(n^2 + 1)}{2} = \frac{n(n^2 + 1)}{2} \quad (5.2)$$

Para $n = 3$ resulta $K_3 = 15$.

Y hay una tercera propiedad, que será la que más usemos. Conocidos dos elementos a y b de una misma línea, el tercero queda determinado sin ambigüedad:

$$c = 15 - (a + b) \quad (5.3)$$

5.3 Ocho y ocho: ¿casualidad?

Detente aquí un momento.

El tablero del tres en raya tiene ocho formas de ganar: tres filas, tres columnas y dos diagonales. El cuadrado mágico de orden tres tiene ocho conjuntos de tres casillas que suman lo mismo: tres filas, tres columnas y dos diagonales.

Ocho y ocho.

Inténtalo primero · Antes de continuar

¿Es casualidad?

Y si no lo es, ¿qué habría exactamente que comprobar para asegurarse? No basta con que los números coincidan: dos conjuntos pueden tener el mismo tamaño y no tener nada que ver.

Piénsalo antes de pasar a la sección siguiente. La respuesta es más fuerte de lo que parece a primera vista.

5.4 La correspondencia exacta

Numeremos las casillas del tablero de 0 a 8, por filas, y definamos la aplicación que a cada casilla le asigna su número en la matriz de referencia:

$$\mu : \{0, \dots, 8\} \longrightarrow \{1, \dots, 9\}, \quad \mu(t) = M_{[t/3], t \bmod 3} \quad (5.4)$$

Como los nueve valores son distintos, μ es una biyección: nada se pierde al traducir, y se puede volver.

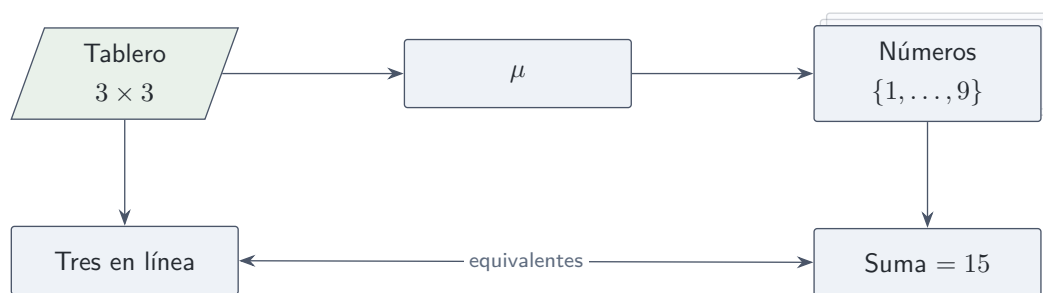


Figura 5.1: La biyección μ traslada el tablero a un conjunto de números. Bajo ella, la condición geométrica de victoria se convierte en una condición aritmética. El camino de abajo es el que interesa: son la misma cosa dicha de dos maneras.

Ahora sí, la respuesta al reto.

Proposición 5.2 (Correspondencia de líneas). Sea $\mathcal{L}$ el conjunto de las ocho líneas ganadoras del tres en raya y sea $\mathcal{S} = \{S \subset \{1, \dots, 9\} : |S| = 3, \sum_{s \in S} s = 15\}$. Entonces μ induce una biyección entre $\mathcal{L}$ y $\mathcal{S}$.

De los $\binom{9}{3} = 84$ subconjuntos de tres elementos de $\{1, \dots, 9\}$, exactamente ocho suman quince:

$$\mathcal{S} = \{\{1, 5, 9\}, \{1, 6, 8\}, \{2, 4, 9\}, \{2, 5, 8\}, \{2, 6, 7\}, \{3, 4, 8\}, \{3, 5, 7\}, \{4, 5, 6\}\} \quad (5.5)$$

Aplicando μ a las ocho líneas del tablero y comparando conjuntos se obtiene exactamente $\mathcal{S}$. No hay línea sin trío ni trío sin línea. $\square$

💡 Detengámonos a pensar · Lo que acaba de pasar

No hemos encontrado un truco que funcione «casi siempre». Hemos encontrado una **equivalencia exacta**.

De ochenta y cuatro tríos posibles, exactamente ocho suman quince, y esos ocho son precisamente las ocho maneras de ganar. Ni uno de más, ni uno de menos.

La consecuencia da sentido a todo lo que viene después: *el algoritmo no necesita conocer la geometría del tablero*. Para saber si alguien ha ganado no hay que recorrer filas ni diagonales. Basta comprobar si tiene tres números que sumen quince.

💡 Detengámonos a pensar · Una aclaración que este libro debe hacer

Esta correspondencia **no es un descubrimiento de este libro**, y sería deshonesto dejarte creer lo contrario.

Está documentada desde hace más de medio siglo. John A. Michon la describió formalmente en 1967, al analizar un juego llamado *JAM* —también conocido como *Number Scrabble*, *Pick 15* o *3 to 15*— en el que los jugadores eligen números del uno al nueve y gana quien reúna tres que sumen quince (Michon, 1967). Ese juego, demostró, es el tres en raya con otro disfraz.

Lo que sí ocurrió, y es lo que cuenta este libro, es que llegué a ella **por mi cuenta**,

sin conocer ese trabajo, buscando una estructura que coincidiera en dimensión con el tablero.

Y aquí está el motivo por el que la aclaración, lejos de restar, suma. Si lo interesante fuera la correspondencia, este libro sobraría: bastaría con leer a Michon. Lo interesante es **el camino**: qué preguntas hay que hacerse para llegar a un sitio así partiendo de un problema que no se sabe resolver.

Ese camino es reproducible aunque el destino ya estuviera en el mapa.

✓ Compruébalo · Que los ocho tríos son exactamente las ocho líneas

Hazlo a mano, que se tarda menos de lo que parece. Escribe las ocho líneas del tablero, tradúcelas con la matriz de la ecuación (5.1), y compara con la lista de la ecuación (5.5). Y si prefieres que lo haga la máquina, este es el prompt.

🤖 Prompt · Comprobar la correspondencia

Considera el cuadrado mágico de orden 3 con esta disposición:

```
2 7 6
9 5 1
4 3 8
```

Haz dos cosas y muéstrame el resultado de cada una:

1. Lista todos los subconjuntos de tres números distintos del 1 al 9 cuya suma sea exactamente 15. Dime cuántos hay.
2. Toma las ocho líneas ganadoras del tres en raya (3 filas, 3 columnas y 2 diagonales), sustituye cada casilla por el número que ocupa esa misma posición en el cuadrado de arriba, y dime qué conjunto de tres números corresponde a cada línea.

Después compara las dos listas y dime si coinciden exactamente, si sobra alguna en una de ellas, o si falta alguna.

5.5 Las simetrías de ambos objetos

El cuadrado admite las ocho transformaciones del grupo diédrico D_4 : cuatro rotaciones y cuatro reflexiones. Aplicadas a M producen las ocho disposiciones posibles del cuadrado mágico de orden tres.

Esto importa por dos motivos. Primero, porque la elección concreta de M es arbitraria: cualquiera de las ocho serviría. Y segundo, porque el mismo grupo actúa sobre el tablero del tres

en raya. Los dos objetos comparten simetrías, lo cual ya es indicio de que la correspondencia no es superficial.

5.5.1 Lo que la transformación conserva

Una transformación solo sirve si conserva lo que hace que el problema sea ese problema. Aquí se conservan tres cosas.

La condición de victoria. Por la proposición 5.2, ganar y poseer un trío de suma quince son la misma condición.

La disponibilidad de casillas. Una casilla está libre si y solo si su número no lo tiene nadie.

La estructura posicional. El tablero distingue centro, esquinas y lados, y esa distinción importa para jugar bien. Bajo μ no se pierde: se vuelve aritmética.

$$\text{centro} = \{5\}, \quad \text{esquinas} = \{2, 4, 6, 8\}, \quad \text{lados} = \{1, 3, 7, 9\} \quad (5.6)$$

El centro es el cinco. Las esquinas son los pares. Los lados, los impares. Compruébalo sobre la ecuación (5.1): es inmediato, y no tuve que añadirlo. Estaba ahí, y no se veía.

La correspondencia queda establecida, y con ella todo lo que hace falta saber. Falta lo otro: convertirla en un programa. El capítulo siguiente lo construye por etapas, y cada una empieza con una pregunta para que la intentes antes de leer la respuesta.

Capítulo 6

Construyendo el algoritmo, etapa por etapa

Cinco etapas, y en cada una la misma secuencia: la pregunta, tu intento, la respuesta, y una pausa para mirar qué acaba de ocurrir.

No te saltes los retos. El rato de desconcierto entre la pregunta y la respuesta es donde ocurre lo que este libro quiere enseñarte; si vas directo a la solución, habrás leído un algoritmo y no habrás practicado nada.

6.1 Etapa 1: representar sin coordenadas

Con la biyección μ del capítulo anterior, una partida deja de ser una secuencia de coordenadas y pasa a ser un reparto de los números 1 a 9 entre dos jugadores. El estado queda descrito por el par (P, R) : los números en poder propio y los del rival, con $P \cap R = \emptyset$. Las casillas libres son $\{1, \dots, 9\} \setminus (P \cup R)$.

Qué es «el estado de la partida»

Pensamiento tradicional

Una matriz de tres por tres con marcas. Para saber algo de ella hay que recorrerla: mirar filas, columnas y diagonales, comparar ocupantes, contar. La posición de cada marca importa, y la posición se expresa en dos coordenadas.

Transformación estructural

Dos conjuntos de números. Para saber algo de ellos hay que sumar. No hay coordenadas, no hay filas ni columnas. Un jugador *tiene* el 2, el 7 y el 6, y esos tres números dicen por sí solos todo lo que hay que saber.

Falta una segunda decisión, independiente de la anterior: cómo anotar quién ocupa qué, de modo que una línea pueda examinarse de un vistazo.

Inténtalo primero · Etapa 1

Necesitas saber, mirando una línea, cuántas fichas tiene cada jugador en ella. Se te ocurrirá contar. Pero hay una forma de que la *suma* de la línea te lo diga directamente, sin contar nada. ¿Qué dos números asignarías a las fichas para que eso funcione? ¿Y qué condición tienen que cumplir entre sí?

La respuesta: casilla libre vale 0, ficha propia 1, ficha del rival 10.

$$S = a + 10b, \quad a + b \leq 3 \quad (6.1)$$

donde a es el número de casillas propias en la línea y b el de casillas del rival. Como en

una línea caben tres fichas, $a \leq 3 < 10$, y entonces la suma es una **codificación posicional**: la cifra de las unidades cuenta tus fichas y la de las decenas las del rival.

Leer 20 es leer «dos del rival, ninguna mía». Leer 2 es leer «dos mías». No hay que interpretar: se lee.

💡 Detengámonos a pensar · Por qué diez y no otro número

La condición real es que ninguna suma pueda confundirse con otra. El menor número que la cumple no es diez, sino **cuatro**: como $a, b \leq 3$, la expresión $a + 4b$ ya distingue todos los casos.

Entonces, ¿por qué diez? Porque diez hace que la suma se lea en base decimal sin conversión mental.

Es una elección por *comprensión*, no por necesidad. Aparece aquí en pequeño el criterio que gobierna todo el libro: entre dos representaciones igualmente correctas, se prefiere la que se entiende.

👤 Prompt · Etapa 1: representar el tablero

Vamos a construir juntos, paso a paso, un algoritmo que juegue al tres en raya. Iré pidiéndote una etapa cada vez. No te adelantes a las siguientes.

Etapa 1. Necesito representar el estado de la partida así:

- Cada jugador posee un conjunto de números del 1 al 9.
- Los números salen de este cuadrado mágico, donde cada casilla del tablero se corresponde con el número de su misma posición:

```
2 7 6
9 5 1
4 3 8
```

Escríbeme el código que represente ese estado y que sepa decir qué números quedan libres. Nada más por ahora.

Explicame en dos frases por qué esta representación no necesita coordenadas.

👋 Sin programar · Etapa 1

No hace falta que ejecutes nada. Coge papel y dibuja el cuadrado mágico. Cada vez que alguien juegue, tacha el número de esa casilla y anótalo en la lista del jugador correspondiente.

Eso es exactamente lo que hará el programa. Si puedes seguirlo en papel, entiendes el algoritmo.

6.2 Etapa 2: saber quién ha ganado

Inténtalo primero · Etapa 2

Un jugador tiene los números 2, 5 y 8. ¿Ha ganado?

Ahora tiene 2, 5 y 9. ¿Ha ganado?

¿Cómo lo has decidido? Escríbelo como una regla que funcione siempre, sin mirar ningún tablero.

Código 6.1: Condición de victoria, sin geometría alguna.

```
1 def es_linea(casillas):
2     """Tres casillas forman recta si y solo si suman la constante magica."""
3     return len(casillas) == 3 and sum(casillas) == CONSTANTE_MAGICA
4
5 def tiene_linea(mias):
6     return any(es_linea(trio) for trio in combinations(mias, 3))
```

Comprobar si alguien ha ganado

Pensamiento tradicional

Recorrer las ocho líneas. Para cada una, mirar sus tres casillas y comprobar si las tres tienen la misma marca.

Hace falta la lista de las ocho líneas, escrita a mano en alguna parte del programa.

Transformación estructural

Tomar los números que tiene el jugador y comprobar si tres de ellos suman quince. No hace falta ninguna lista. Las ocho líneas no están escritas en el programa: *son* los ocho tríos que suman quince.

Prompt · Etapa 2: detectar la victoria

Etapa 2. Ahora quiero detectar si un jugador ha ganado.

En el cuadrado mágico, las ocho líneas ganadoras del tres en raya son exactamente los ocho conjuntos de tres números del 1 al 9 que suman 15.

Escribe una función que reciba los números que posee un jugador y devuelva si ha ganado, comprobando si tres cualesquiera de ellos suman 15. No uses ninguna lista de líneas ni coordenadas del tablero.

Después dime cuántas líneas has tenido que escribir explícitamente en el código, y compáralo con las que necesitarías si trabajaras sobre la cuadrícula.

6.3 Etapa 3: ver las amenazas

Definición 6.1 (Amenaza). Un jugador *amenaza* una línea cuando posee dos de sus tres casillas y la tercera está libre.

Inténtalo primero · Etapa 3

Tu rival tiene el 2 y el 6. ¿Debes preocuparte? Si la respuesta es sí, ¿dónde tienes que jugar?

Contesta sin dibujar el tablero.

Por la ecuación (5.3), si posee a y b , la casilla que cierra la línea es $15 - (a + b)$. Con 2 y 6: $15 - 8 = 7$. Hay amenaza si el 7 sigue libre, y la jugada obligada es el 7.

Código 6.2: Casillas que cierran una recta propia.

```
1 def cierres(mias, libres):
2     """Casillas libres que completan una recta propia."""
3     return sorted(
4         falta
5         for a, b in combinations(sorted(mias), 2)
6         if (falta := CONSTANTE_MAGICA - a - b) in libres
7     )
```

Fíjate en lo que *no* hay en ese código: no se comprueba si a y b están en una misma línea. No hace falta. Por la proposición 5.2, si $15 - (a + b)$ cae en el rango y los tres son distintos, entonces el trío *es* una línea. La propiedad matemática sustituye a la comprobación.

Detengámonos a pensar · Esto es lo que significa «transformar»

Una comprobación estructural —¿están estas tres casillas alineadas?— ha desaparecido del programa. No se ha optimizado: ha dejado de ser necesaria, porque la representación la garantiza.

Cuando busques transformaciones en otros problemas, esta es la señal: no que el código sea más corto, sino que haya comprobaciones que ya no haga falta hacer.

Prompt · Etapa 3: detectar amenazas

Etapa 3. Ahora las amenazas.

Un jugador amenaza cuando tiene dos números de una línea y el tercero sigue libre. Como toda línea suma 15, si tiene los números a y b , el que le falta es $15 - (a + b)$.

Escribe una función que reciba los números de un jugador y los números aún libres, y devuelva la lista de casillas que ese jugador podría usar para

completar una línea.

Importante: no compruebes si a y b pertenecen a una misma línea. Explícame por qué esa comprobación es innecesaria aquí.

6.4 Etapa 4: decidir dónde jugar

Ya sabemos detectar. Falta decidir.

🔗 Inténtalo primero · Etapa 4

Tienes toda la información: sabes si puedes ganar, sabes si te pueden ganar.

Escribe las reglas que decidan dónde jugar, ordenadas por prioridad. ¿Cuántas te salen? ¿En qué orden?

Y una pregunta más difícil: ¿estás seguro de que con esas reglas no pierdes nunca?

Las reglas son cuatro, y el orden importa tanto como las reglas mismas.

Regla 1. Ganar si es posible. Si hay una casilla que completa una línea propia, se juega ahí. Parece obvia, pero debe enunciarse con prioridad explícita: sin ella, un algoritmo orientado a no perder puede bloquear una amenaza ignorando una victoria disponible.

Regla 2. Si se puede perder, no perder. Si el rival amenaza, se ocupa la casilla que cierra su línea. Es la misma operación que la anterior, aplicada al otro conjunto.

Regla 3. Evitar casos dobles. Si el rival puede alcanzar una posición con dos amenazas a la vez, bloquear una dejaría la otra abierta. El único recurso es atacar, para forzar su respuesta. Pero atacar no basta: hay que atacar donde su bloqueo obligado *no* le construya la doble amenaza que se quiere evitar.

Regla 4. Preferencia posicional. Si nada de lo anterior se aplica, se juega el centro; en su defecto, una esquina; en su defecto, un lado. Por la ecuación (5.6): el cinco, un par, un impar.

💡 Detengámonos a pensar · Una honestidad necesaria

Las reglas 1 y 2 nacen de la transformación: son sumas y restas sobre los números.

Las reglas 3 y 4 no. Son *política de juego*, conocimiento del tres en raya que la transformación no suministra. Lo que sí hace la transformación es abaratar su comprobación y permitir enunciarlas sin geometría.

Decirlo importa. Un libro que afirmara que el cuadrado mágico resuelve el juego por sí solo estaría exagerando, y quien lo implementara lo descubriría en cuanto perdiera una partida.

6.5 Etapa 5: las cuatro reglas completas

Código 6.3: Las cuatro reglas, en orden de prioridad.

```

1  def jugada(mias, suyas):
2      """Casilla en la que jugar. Cuatro reglas, en orden de prioridad."""
3      libres = CASILLAS - set(mias) - set(suyas)
4
5      # Regla 1. Ganar si es posible.
6      if propias := cierres(mias, libres):
7          return propias[0]
8
9      # Regla 2. Si se puede perder, no perder.
10     if ajenas := cierres(suyas, libres):
11         return ajenas[0]
12
13     # Regla 3. Evitar casos dobles.
14     if _amenaza_doble_en_camino(suyas, libres):
15         if ataque := _mejor_ataque(mias, suyas, libres):
16             return ataque
17
18     # Regla 4. Preferencia posicional: centro, esquinas, lados.
19     for nivel in PREFERENCIA:
20         if disponibles := sorted(nivel & libres):
21             return disponibles[0]
22
23     raise ValueError("no quedan casillas libres")

```

A las cuatro reglas se añaden tres invariantes de la partida, que no dicen dónde jugar sino qué hace válida una secuencia de jugadas: en cada turno se juega una sola vez, cada jugador emplea siempre la misma ficha, y se respeta la alternancia.

Prompt · Etapa 5: las cuatro reglas

Etapa 5. Ahora la decisión. Implementa estas cuatro reglas, aplicadas estrictamente en este orden:

1. Si puedo completar una línea propia, juego ahí.
2. Si el rival puede completar la suya, juego ahí para bloquearlo.
3. Si el rival puede alcanzar una posición con DOS amenazas a la vez, ataco creando una amenaza propia. Pero elijo el ataque cuyo bloqueo obligado del rival no le genere esas dos amenazas.
4. Si nada de lo anterior aplica: juego el 5; si no está libre, un número par; si tampoco, un número impar.

Recuerda que en esta representación el 5 es el centro, los pares son las

esquinas y los impares son los lados.

Cuando termines, dime por qué el orden de las reglas 1 y 2 no puede invertirse, y dame un ejemplo concreto donde invertir las haría perder la partida.

6.6 Una partida entera, sin dibujar el tablero

Sigamos una partida completa. El rival abre; el algoritmo responde.

Apertura. $P = \emptyset$, $R = \emptyset$. El rival toma una esquina, el 2. Ahora $R = \{2\}$.

Respuesta. Turno del algoritmo. Ninguna línea tiene dos casillas del mismo jugador, así que las reglas 1 y 2 no se aplican. El rival tampoco puede construir una doble amenaza en una sola jugada. Actúa la regla 4: el centro está libre, se juega el 5. Queda $P = \{5\}$.

Ataque. El rival toma el 8, la esquina opuesta: $R = \{2, 8\}$.

Observa la aritmética: $2 + 8 = 10$ y $15 - 10 = 5$, que es del algoritmo. La línea del rival está muerta. Lo hemos sabido con una resta, sin mirar el tablero.

Turno del algoritmo. No hay victoria ni amenaza que bloquear, pero el rival, jugando en una esquina libre, alcanzaría dos amenazas simultáneas. Se dispara la regla 3: hay que atacar, y por un lado, no por una esquina. El algoritmo juega el 1, y queda $P = \{5, 1\}$. Eso amenaza, porque $15 - (5 + 1) = 9$ y el 9 está libre.

Defensa. El rival está obligado a bloquear el 9: $R = \{2, 8, 9\}$. Al hacerlo no ha construido doble amenaza, que era justamente el objeto de la regla 3.

Final. La partida sigue con las reglas 1 y 2 alternándose hasta agotar las casillas. En esta línea concreta, tablas.

💡 Detengámonos a pensar · Relee la partida

Vuelve al principio de esta sección y fíjate en una cosa: no ha aparecido ni una sola vez la palabra «fila», «columna» ni «diagonal».

La partida entera se ha jugado con sumas y restas. Y sin embargo es la misma partida que habrías jugado en un tablero de papel.

El algoritmo está construido: cinco etapas, cuatro reglas y tres invariantes. Y en la partida que acabamos de seguir, funciona.

Eso es exactamente lo que no basta. Una partida que sale bien no dice nada sobre las que no hemos jugado, y el encargo no era jugar bien: era *no poder perder*. El capítulo siguiente se ocupa de la diferencia entre las dos cosas, que es más grande de lo que parece.

Capítulo 7

Comprobarlo de verdad

El encargo era concreto: la máquina no puede perder. Ni una vez.

Tenemos un algoritmo que parece cumplirlo. Este capítulo trata de la diferencia entre *parecer* y *cumplir*, que es más grande de lo que se suele admitir.

7.1 Jugar unas partidas no demuestra nada

La tentación, cuando el programa funciona, es jugar diez partidas, ganar o empatar las diez, y darlo por bueno.

🕒 Inténtalo primero · Antes de continuar

Has jugado cincuenta partidas contra tu algoritmo y no ha perdido ninguna.

¿Qué has demostrado exactamente?

Y una pregunta más incómoda: si tuviera un fallo que apareciera en una de cada mil partidas, ¿cuántas tendrías que jugar para encontrarlo? ¿Y estarías seguro entonces?

Cincuenta partidas ganadas demuestran que el algoritmo no falla *en esas cincuenta*. Nada más. El problema de las pruebas por muestreo es que su resultado negativo no informa: no encontrar un fallo y no haberlo son cosas distintas.

💡 Detengámonos a pensar · Una ventaja poco frecuente

El tres en raya tiene una propiedad que casi ningún problema real tiene: **se lo puede explorar completo**. El juego entero, con todas las partidas posibles, puede recorrerse en un ordenador cualquiera en menos de un segundo.

Cuando eso ocurre, probar por muestreo es una elección, no una necesidad. Y es la elección equivocada.

7.2 Recorrer el juego entero

La comprobación correcta consiste en explorar el árbol completo: todas las respuestas posibles del rival, en las dos posiciones de salida, y para cada una comprobar cómo termina.

Hay un detalle que conviene no pasar por alto. Si el algoritmo elige al azar entre varias jugadas, «no perder nunca» exige que se cumpla para **todas** las elecciones posibles, no para la que salió esta vez. Por eso el recorrido debe abrir también las ramas del propio azar.

Prompt · Comprobar que nunca pierde

Vamos a verificar el algoritmo, y no con unas cuantas partidas de prueba.

Recorre el árbol COMPLETO del juego: todas las respuestas posibles del rival, en las dos posiciones de salida (empezando el algoritmo y empezando el rival). Cuenta partidas jugadas, victorias, empates y derrotas.

Si mi algoritmo elige al azar entre varias jugadas igual de buenas, explora también todas esas alternativas: quiero saber si pierde en ALGUNA rama posible, no solo en la que salga.

El objetivo es cero derrotas. Si aparece aunque sea una, muéstrame la partida que lleva a ella e identifica qué regla falló.

Sin programar · Qué estás pidiendo aquí

No necesitas entender el código para entender la comprobación. Estás pidiendo que se jueguen *todas* las partidas que podrían jugarse, y que se cuente en cuántas pierde el algoritmo.

Si el resultado es cero, no es que probablemente no pierda: es que no puede perder.

7.3 La regla que estaba mal y cómo la descubrí

Aquí viene la parte interesante, y la razón por la que este capítulo existe.

El algoritmo original tenía cuatro reglas, y la cuarta decía: *si ninguna de las anteriores se aplica, juega al azar en cualquier posición libre*.

Parecía inofensiva. Al fin y al cabo, si no hay nada que ganar ni que bloquear, ¿qué más da dónde juegues?

Cuando recorrí el árbol completo, el resultado fue este:

Cuadro 7.1: Resultado con la regla 4 entendida como azar puro.

Configuración	Partidas	Derrotas
Reglas 1, 2, 4 aleatoria	86 048	8 320
Reglas 1, 2, 3, 4 aleatoria	39 232	1 600

El algoritmo perdía. Y no por un fallo táctico: perdía *antes* de que hubiera nada que detectar. Jugando al azar en las primeras jugadas podía ocupar un lado en vez del centro y entregar una posición ya perdida, cuando todavía ninguna regla tenía trabajo que hacer.

La corrección resultó ser mínima: sustituir el azar puro por una preferencia posicional, conservando el azar dentro de cada nivel.

Cuadro 7.2: Resultado con la regla 4 corregida.

Configuración	Partidas	Derrotas
Reglas 1, 2 + 4 posicional (sin regla 3)	2 096	64
Reglas 1, 2, 3 + 4 posicional	1 784	0

💡 Detengámonos a pensar · Lo que este hallazgo enseña

Dos cosas, y ninguna trata del tres en raya.

La primera: un algoritmo puede parecer correcto durante miles de partidas y no serlo.

La regla del azar habría pasado cualquier prueba razonable por muestreo.

La segunda, y más importante: **esto no es un tropiezo del relato, es el resultado.**

La historia original enunciaba cuatro reglas y la comprobación mostró que una estaba mal especificada. Que se detecte y se corrija es exactamente lo que separa una investigación de una demostración de algo que ya se daba por bueno.

Si este libro hubiera omitido este episodio, sería un libro peor y menos fiable.

7.4 Cero derrotas: qué significa exactamente

Con las cuatro reglas corregidas, el recorrido completo da **cero derrotas en 1 784 partidas**, cubriendo todo el juego posible del rival y todo el margen de azar propio, empezando y respondiendo.

Conviene ser preciso sobre qué se ha demostrado y qué no.

Lo que se ha demostrado. Que no existe ninguna secuencia de jugadas del rival, ni ninguna elección del azar propio, que lleve al algoritmo a perder. La condición del encargo se cumple, y se cumple con certeza, no con alta probabilidad.

Lo que no se ha demostrado. Que el algoritmo gane siempre que podría. No es lo que se pedía. Un rival que juegue bien forzará tablas, como ocurre entre dos jugadores humanos competentes.

Lo que tampoco se ha demostrado. Que las cuatro reglas sean las únicas posibles, ni las mejores. Solo que bastan.

7.5 Los errores que conviene provocar a propósito

Si has implementado el algoritmo, merece la pena romperlo deliberadamente. Se aprende más de los fallos que se provocan que de los que se evitan.

✅ Compruébalo · Tres roturas instructivas

1. Invierte las reglas 1 y 2. Que bloquee antes de ganar. Busca la partida donde tiene la victoria servida y la desaprovecha para defender algo que ya no importa.

2. Devuelve la regla 4 al azar puro. Verás reaparecer las derrotas, y verás que ocurren en las primeras jugadas, no en las últimas.

3. Quita la regla 3. Aparecen 64 derrotas, y todas caen en las 1 624 partidas —de las 2 096 del recorrido— en que el algoritmo responde en segundo lugar. Todas son además la misma historia: el rival construye dos amenazas y solo se puede bloquear una.

Con esto el algoritmo está construido y comprobado. Cumple lo que se le pidió y sabemos exactamente en qué sentido lo cumple.

Quedaría celebrarlo. En vez de eso, el capítulo siguiente le hace la pregunta más incómoda que se le puede hacer: si el cuadrado mágico, que ha dado nombre a todo esto, era realmente necesario.

Capítulo 8

¿Era necesario el cuadrado mágico?

Este capítulo formula la objeción más seria que puede hacersele a todo lo anterior. La formula el libro contra sí mismo, porque si nadie la hace, el lector la hará por su cuenta y con razón.

8.1 La objeción del lector curioso

Vuelve un momento a la etapa 3. Detectábamos las amenazas sumando las líneas con las fichas 1 y 10, y localizábamos la casilla con $15 - (a + b)$.

Ahora fíjate en algo. Cuando una línea está amenazada, tiene exactamente una casilla vacía. Una sola.

🔗 Inténtalo primero · La pregunta incómoda

Si la línea amenazada tiene una única casilla vacía... ¿para qué necesitas el cuadrado mágico para encontrarla?

Podrías simplemente jugar en la casilla que está vacía. Ya la tienes localizada: es la que estabas mirando.

Entonces, ¿hace falta el cuadrado mágico, o basta con las fichas 1 y 10?

Piénsalo antes de seguir. Y si crees que sí hace falta, intenta decir para qué exactamente.

8.2 Tres versiones del mismo algoritmo

La pregunta no se responde opinando. Se responde construyendo las tres versiones posibles y comparándolas.

Cuadro 8.1: Las tres formulaciones y lo que cada una necesita.

	Detecta con	Localiza con	Geometría que usa
V1	Suma de las 8 rectas	La casilla vacía	Rectas, coordenadas, posiciones
V2	Suma de las 8 rectas	Complemento a 15	Rectas, coordenadas, posiciones
V3	Tríos que suman 15	El número que falta	Ninguna

V2 es la versión del relato original. V1 es la misma sin cuadrado mágico. V3 es la versión que hemos construido en este libro: sin tablero, sin lista de rectas y sin coordenadas.

8.3 La respuesta: sí, se puede quitar

Sometí las tres al recorrido exhaustivo del juego.

Cuadro 8.2: Verificación exhaustiva de las tres formulaciones.

Formulación	Partidas	Derrotas
V1 — fichas 1/10, sin cuadrado mágico	551	0
V2 — fichas 1/10 con cuadrado mágico	551	0
V3 — solo aritmética, sin geometría	551	0

Ninguna pierde. Y hay más: V1 y V2 **toman exactamente la misma decisión en las 723 posiciones comparadas**, sin una sola discrepancia.

La respuesta, entonces, es **sí**. En la formulación del relato original, el cuadrado mágico puede retirarse sin que nada cambie.

 Detengámonos a pensar · Quién estaba haciendo el trabajo

Esto obliga a corregir algo que quizá dabas por supuesto, y que yo mismo daba por supuesto.

Quien reduce el problema son **las fichas 1 y 10**, no el cuadrado mágico. La codificación posicional convierte «¿hay dos fichas más y un hueco en esta recta?» —una comprobación estructural sobre tres casillas— en «¿esta recta vale 2?» —una comparación con un número—.

El caso contiene, en realidad, **dos transformaciones encadenadas** y no una. Y estaban haciendo cosas distintas.

8.4 Y sin embargo, qué se pierde al quitarlo

Si el cuadrado mágico se retira, el algoritmo sigue funcionando pero conserva toda la geometría: la lista de las ocho rectas escrita a mano, las coordenadas y la clasificación de centro, esquinas y lados.

Es en V3 donde el cuadrado mágico rinde, porque allí se lleva hasta el final y la representación original puede descartarse entera.

Cuadro 8.3: Lo que desaparece cuando la transformación es completa.

Concepto del juego	Expresión aritmética
Ganar	Poseer tres números que sumen 15
Amenazar	Tener a, b con $15 - (a + b)$ libre
Bloquear	Tomar el $15 - (a + b)$ del rival
Centro	El 5
Esquinas	Los números pares
Lados	Los números impares

La lista de las ocho rectas no está en ninguna parte del programa, porque las ocho rectas *son* los ocho tríos que suman quince. Y la clasificación posicional tampoco, porque es la paridad de los propios números.

Nota al lector curioso

El lector curioso se preguntará, y seguramente afirmará, que el cuadrado mágico puede retirarse en el momento de optimizar el algoritmo. La respuesta es **sí**. Comprobado sobre el árbol completo del juego, una implementación que sume las ocho rectas con la codificación de fichas y juegue en la casilla vacía de la recta detectada toma exactamente las mismas decisiones, en las setecientas veintitrés posiciones comparadas, y tampoco pierde nunca.

Sin embargo, para el análisis y el entendimiento del algoritmo y del juego, el cuadrado mágico proporciona una visión posicional clara de los casos de uso, que muchas veces el programador no considera a primera vista al resolver un problema, pero que sí resulta notoria en el momento de optimizar.

De todas formas, esto describe el proceso creativo que quiero mostrar en este libro: la premisa de pensar fuera de la caja conduce a soluciones que a menudo admiten optimización posterior, pero que constituyen un primer y gran acercamiento, por mucho, muy interesante.

8.5 Una transformación parcial no rinde

De todo esto sale el criterio más útil del libro, y conviene enunciarlo aparte porque se aplica lejos del tres en raya.

💡 Detengámonos a pensar · El principio

Una transformación parcial no rinde.

Cuando la representación nueva se superpone a la antigua sin sustituirla, se paga su coste y no se cobra su beneficio. Eso es exactamente V2: dos representaciones conviviendo, y ninguna de las dos pudiendo retirarse.

La ventaja aparece cuando la transformación es **completa** y la representación original puede descartarse.

Y de ahí sale una medida: *una transformación se mide por lo que permite eliminar, no por lo que añade.*

✓ **Compruébalo · Aplícalo a algo tuyo**

Piensa en la última vez que introdujiste una estructura nueva en un programa —una clase, un índice, una caché, una abstracción—.

¿Qué pudiste borrar después? Si la respuesta es «nada», probablemente añadiste una capa en lugar de transformar el problema.

Hemos terminado el taller. Tenemos un algoritmo, sabemos que no pierde, y sabemos qué parte del mérito corresponde a cada cosa.

Falta lo que da sentido a haberlo construido: qué cambió de verdad, qué procedimiento puede extraerse de aquí para un problema que no sea este, y cuándo ese procedimiento no sirve. De eso trata la parte siguiente.

IV

Lo que realmente cambió

Capítulo 9

Menos código, menos reglas, más entendimiento

Hemos construido el algoritmo, lo hemos verificado y hemos comprobado que el cuadrado mágico solo rinde cuando se lleva hasta el final.

Toca ahora la pregunta que da nombre a esta parte: **¿qué cambió realmente?**

Este capítulo responde con el sentido de las cifras. El capítulo 14 trae las tablas completas, el método y las salvedades; aquí interesa qué significan.

9.1 Las cifras, medidas y no estimadas

Antes de mirar ningún número, una aclaración que debería ser innecesaria y no lo es: ninguna cifra de este libro es una estimación. Cómo se midió cada una, contra qué se comparó y con qué salvedades está en el capítulo 14.

💡 Detengámonos a pensar · Por qué insisto en esto

Cualquiera puede escribir un libro afirmando que su método reduce el código cincuenta veces. Basta comparar una versión cuidada propia contra una versión descuidada ajena. La diferencia entre eso y esto es que aquí puedes reproducirlo. El tres en raya lo puedes recorrer completo en cualquier ordenador, y el capítulo 14 trae el prompt para que lo compruebes tú.

Si un libro te da cifras y no te da la manera de comprobarlas, tienes derecho a desconfiar. Incluso de este.

9.2 Cuatro reglas frente a cuatro mil quinientos casos

Aquí está el resumen de todo el trabajo.

Cuadro 9.1: Las tres soluciones al mismo problema.

	Casos	Condicionales	Líneas de código
Enumeración exhaustiva	4 520	4 520	4 522
Con simetría	627	627	630
Cuadrado mágico	4 reglas	14	77

Cincuenta y nueve veces menos código que la primera. Ocho veces menos que la segunda, que es la mejor optimización conocida del enfoque tradicional.

Pero fíjate en la columna de la izquierda, que es la que importa.

 Detengámonos a pensar · La reducción no es de tamaño. Es de naturaleza

Pasar de 4520 casos a 627 es una reducción **cuantitativa**. Sigues teniendo una lista; es más corta.
Pasar de 627 casos a cuatro reglas no es lo mismo. **Ya no hay lista**. Hay un principio y cuatro decisiones que se derivan de él.
Es la diferencia entre resumir un mapa y entender el terreno. Con el mapa resumido llegas a los sitios que están dibujados. Con el terreno entendido llegas también a los que no.

 Qué pasa si aparece una situación nueva

Con una lista de casos

La lista no la contempla. Hay que añadir un caso.
Y hay que asegurarse de que el caso nuevo no contradice a ninguno de los seiscientos veintisiete anteriores, cosa que nadie puede comprobar leyendo.

Con cuatro reglas

Las cuatro reglas la cubren o no la cubren, y se sabe cuál de las dos cosas porque se puede razonar.
Si no la cubren, se añade una quinta regla y se comprueba que el conjunto sigue funcionando. Cinco reglas se pueden tener en la cabeza a la vez.

9.3 Defender un programa leyéndolo

Y llegamos a lo que buscaba desde el principio, cuando dije que necesitaba una solución sobre la cual pudiera tener completo entendimiento *para garantizar las afirmaciones que de ella deriven*.

Conviene separar dos cosas que suelen ir juntas bajo la palabra «entender».

Cuadro 9.2: Dos preguntas distintas sobre un mismo programa.

	Lista de casos	Cuatro reglas
¿Por qué hizo <i>esta</i> jugada?	sí	sí
¿Por qué juega bien <i>siempre</i> ?	no	sí

La primera pregunta la responden las dos soluciones. Se busca el caso, se lee, y ya está: hizo eso porque el caso 3417 dice eso.

La segunda es la que separa una solución de la otra. En la lista, la respuesta está repartida entre cuatro mil quinientos sitios, y **nadie la tiene entera**. Ni quien la escribió. La corrección del conjunto se estableció probando, no comprendiendo.

En las cuatro reglas, la respuesta cabe en un párrafo: las líneas ganadoras son exactamente los tríos que suman quince, luego detectar amenazas es restar; sobre esa detección, cuatro

reglas en orden de prioridad bastan para no perder, y eso está comprobado recorriendo el juego entero.

 Detengámonos a pensar · Qué significa «defender»

Significa poder responder ante alguien que pregunta.

Imagina que este programa gobierna algo que importa, y alguien te pide explicaciones. Con la lista, lo máximo que puedes decir es «funciona, lo hemos probado mucho». Con las cuatro reglas puedes explicar por qué, y quien te escucha puede comprobarlo por su cuenta.

Esa diferencia no aparece en ninguna medida de rendimiento. Y en muchos contextos es la única que importa.

 Sin programar · Lo mismo, fuera de la programación

Un procedimiento de oficina de ochenta páginas con un caso para cada situación, frente a cuatro criterios de los que se derivan las decisiones.

El primero funciona mientras la realidad se parezca a lo previsto, y nadie puede defenderlo entero. El segundo se puede explicar a alguien nuevo en diez minutos, y esa persona sabrá qué hacer ante algo no previsto.

Es la misma diferencia, y no tiene nada que ver con ordenadores.

9.4 Lo que la transformación no hizo

Ahora la parte que un libro entusiasta se saltaría.

No suprimió las reglas. Quedaron cuatro, y ya quedó advertido en la etapa 4 por qué el cuadrado mágico no las suprime.

Dos de esas cuatro reglas no vienen de la transformación. Evitar dobles amenazas y preferir el centro son conocimiento del tres en raya, no consecuencia de la aritmética. La transformación las deja enunciar en una línea cada una y comprobar con una resta, pero no las descubre por ti. No es poco. No es lo mismo.

No hizo el algoritmo más rápido. Ni falta que hacía: el tres en raya es pequeño y cualquiera de las tres soluciones responde al instante. La formulación puramente aritmética incluso cuesta *más* operaciones que la tabla, porque cambia datos por cálculo.

No apareció sola. Costó varias horas de una persona apartada del problema. Ese coste es real y no figura en ninguna tabla.

 Detengámonos a pensar · Lo que sí hizo, dicho con precisión

Reunámoslo en una frase, porque es la conclusión de esta parte del libro:

La transformación no resolvió el problema. Cambió qué había que saber para resolverlo.

Antes había que conocer cuatro mil quinientas situaciones. Después, una propiedad aritmética y cuatro reglas.

El problema era el mismo. La cantidad de cosas que hay que tener en la cabeza para dominarlo, no.

Con eso queda dicho qué cambió en *este* problema, y con qué salvedades.

Pero un caso no es un método. Si lo que ocurrió aquí solo sirve para el tres en raya, no había libro que escribir. El capítulo siguiente intenta lo difícil: sacar de este caso un procedimiento que puedas aplicar a un problema que yo no he visto nunca.

Capítulo 10

El modelo de pensamiento

Todo lo anterior ha ocurrido una vez, en un problema, con una persona.

Este capítulo intenta lo difícil: extraer de ese caso un procedimiento que puedas aplicar a un problema tuyo, que no será el tres en raya y para el que no habrá ningún cuadrado mágico esperando.

💡 Detengámonos a pensar · Una advertencia antes de empezar

Lo que sigue **no es una receta**. No garantiza encontrar nada.

Es un guion de preguntas, ordenado según el orden en que resultaron útiles. Su valor no está en que te lleve a la solución, sino en que impide que te saltes la parte en la que podría aparecer.

Un método que no promete resultados y solo promete no dejarte ir demasiado deprisa parece poca cosa. Es lo que hay, y es más de lo que suele darse.

10.1 Los siete movimientos

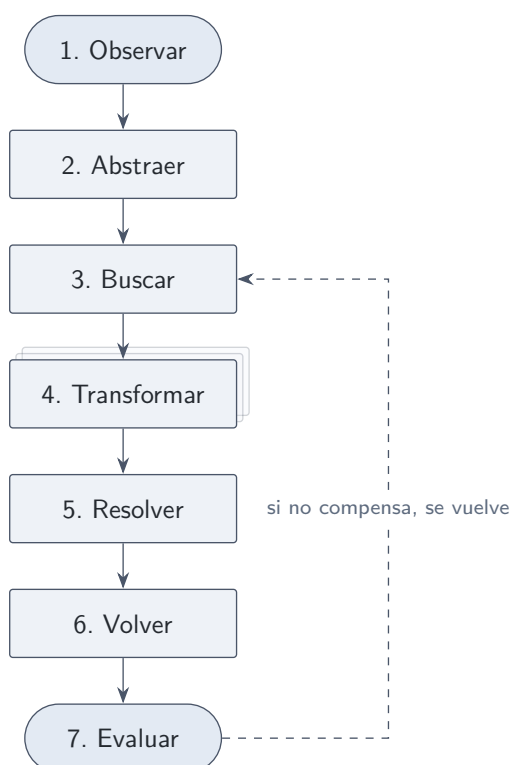


Figura 10.1: Los siete movimientos. La flecha de vuelta no es decorativa: la mayor parte de las veces se recorre, porque la primera estructura que se encuentra no sirve.

Conviene decir desde ya que los movimientos 3 y 4 son donde se fracasa. Los demás son trabajo; esos dos son búsqueda, y la búsqueda a veces no encuentra.

10.2 Observar: separar el problema de su descripción

Pregunta del movimiento: *¿qué es el problema y qué es la manera en que me lo han contado?*

Casi todo problema llega envuelto en una descripción, y la descripción viene con decisiones tomadas que nadie señala como decisiones. «Un tablero de tres por tres» es una descripción. «Nueve posiciones con ocho conjuntos especiales» es otra, del mismo problema.

Qué buscar aquí:

- **Las reglas de verdad.** Qué hay que conseguir, qué está permitido, cuándo se acaba. Esto es el problema.
- **Lo que solo es la forma de contarlo.** Coordenadas, nombres, unidades, el orden en que te lo explicaron, el formato en que llegaron los datos. Esto es la descripción, y es negociable.
- **Lo que no cambia nunca.** Si algo se mantiene invariable pase lo que pase, apúntalo: será lo que haya que conservar al transformar.

🔗 Inténtalo primero · Aplícalo a algo tuyo

Coge un problema real que tengas ahora mismo. Escribe su enunciado tal como te llegó. Ahora subraya todo lo que sea *cómo te lo contaron* y no *qué hay que conseguir*. Suele quedar subrayada más de la mitad.

10.3 Abstraer: quitar lo que no es esencial

Pregunta del movimiento: *¿qué queda si le quito todo lo que no decide?*

Este movimiento es incómodo porque consiste en tirar información. Y tirar información parece siempre un error.

En el tres en raya, abstraer significó dejar de hablar de cruces y círculos, de turnos y de jugadores, y quedarse con: nueve posiciones, ocho conjuntos de tres, gana quien complete uno.

💡 Detengámonos a pensar · La prueba del enunciado desnudo

Hay una manera práctica de saber si has abstraído bastante: intenta describir el problema **sin decir de qué va**.

Si al describirlo tienes que decir «es un juego», «son clientes», «son facturas», todavía estás dentro de la descripción original. Cuando consigas enunciarlo con formas, cantidades y relaciones, has abstraído.

Y solo entonces el movimiento siguiente es posible. No se puede buscar «otra cosa con la misma forma» mientras la forma esté tapada por el asunto.

10.4 Buscar: qué otra estructura tiene esta forma

Pregunta del movimiento: *¿qué otra cosa tiene esta misma forma?*

Este es el movimiento del libro entero, y el único que no se puede sistematizar del todo. Lo que sí puede hacerse es orientar la búsqueda.

Cuadro 10.1: Familias donde suele merecer la pena mirar.

Si tu problema tiene...	Mira hacia...
Posiciones y conjuntos especiales	Combinatoria, cuadrados mágicos, diseños
Cosas conectadas entre sí	Grafos, redes, árboles
Estados que se suceden	Autómatas, máquinas de estados
Cantidades que se conservan	Invariantes, leyes de conservación
Simetrías o repeticiones	Grupos, clases de equivalencia
Elegir sin poder probarlo todo	Orden, cotas, poda
Dos cosas que se corresponden	Bijecciones, isomorfismos

Mi criterio fue la **coincidencia de dimensión**: busqué algo de tres por tres. Es un criterio pobre y funcionó. No hace falta que el criterio sea sofisticado; hace falta que sea *de forma* y no de tema.

 Prompt · Buscar estructuras con la misma forma

Te voy a describir un problema sin decirte de qué va, a propósito.

[Aquí describe tu problema en términos de cantidades, posiciones, relaciones y condiciones. Nada de nombres del dominio.]

No quiero que lo resuelvas. Quiero que me digas qué otras estructuras matemáticas, juegos, o problemas conocidos tienen esta misma forma.

Dame al menos cinco candidatos, aunque parezcan de campos que no tienen nada que ver. Para cada uno dime qué parte de la forma coincide y qué parte no.

No descartes ninguno por raro. Los raros son los que me interesan.

 Detengámonos a pensar · Aquí es donde se fracasa

Y conviene decirlo sin suavizar: la mayoría de las veces este movimiento no encuentra nada útil.

Buscas, aparecen cinco candidatos, ninguno encaja, y vuelves al problema igual que estabas salvo por las horas perdidas.

Eso no significa que el movimiento esté mal hecho. Significa que buscar es buscar. El capítulo 11 se ocupa de cuándo dejar de hacerlo.

10.5 Transformar: qué hay que conservar

Pregunta del movimiento: *¿qué tiene que seguir siendo verdad después de traducir?*

Encontrada una estructura candidata, hay que establecer la correspondencia. Y la correspondencia solo vale si conserva lo que decide.

Tres comprobaciones, las mismas que vimos en el capítulo 4:

1. **Todo estado tiene su traducción, y solo una.** Nada se pierde, nada se duplica.
2. **La condición de éxito se traduce exactamente.** Ni de más ni de menos. En nuestro caso: las ocho líneas son los ocho tríos, sin sobrantes ni faltantes. Si hubieran sido nueve tríos y ocho líneas, la correspondencia habría sido inservible.
3. **Se puede volver.** La solución en el dominio nuevo debe poder traducirse al original.

💡 Detengámonos a pensar · El aviso más útil de este capítulo

Recuerda lo que descubrimos en el capítulo 8:

una transformación parcial no rinde. Si la representación nueva convive con la vieja en lugar de sustituirla, pagas su coste sin cobrar su beneficio.

La pregunta que hay que hacerse aquí no es «¿funciona la traducción?», sino «¿qué puedo borrar ahora?». Si la respuesta es «nada», has añadido una capa, no has transformado nada.

10.6 Resolver: aprovechar lo que la nueva forma regala

Pregunta del movimiento: *¿qué es fácil aquí que allí era difícil?*

Este movimiento suele ser el más corto, y es sospechoso: si la transformación era buena, resolver debería resultar casi decepcionante.

En nuestro caso: detectar amenazas pasó a ser una resta. No tuve que inventar un algoritmo listo; tuve que darme cuenta de que ya no hacía falta.

🔍 Cómo se reconoce que la transformación era buena

Mala señal

Sigues escribiendo comprobaciones parecidas a las de antes, solo que con otros símbolos.

Mala señal: has cambiado la notación, no la representación.

Buena señal

Hay comprobaciones que **desaparecen** del programa. No se optimizan: dejan de ser necesarias, porque la nueva forma las garantiza.

Esa es la señal buena. Y se mide contando lo que ya no está.

10.7 Volver: traducir la solución

Pregunta del movimiento: *¿qué significa esto en el problema de verdad?*

Un resultado en el dominio transformado no es una solución. Es una solución *traducible*. Hay que traducirla, y hay que comprobar que al traducirla sigue siendo válida.

Es el movimiento que más se olvida, porque llega cuando uno ya se siente ganador. Y es donde aparecen los errores más caros: soluciones elegantes que no significaban nada en el problema original.

✓ Compruébalo · Antes de dar nada por bueno

Coge la solución traducida y compruébala **en el problema original**, como si no supieras de dónde salió.

En nuestro caso: verifiqué el algoritmo jugando al tres en raya, no comprobando sumas. La aritmética era el camino; el juego era el juez.

10.8 Evaluar: ¿mereció la pena el rodeo?

Pregunta del movimiento: *¿gané algo, y qué?*

El último movimiento es el que casi nadie hace, porque llega cuando el problema ya está resuelto y apetece pasar a otra cosa.

Cinco preguntas:

Cuadro 10.2: Qué preguntarse al final.

Pregunta	Qué revela
¿Es correcta?	Si no, lo demás sobra
¿Qué pude borrar?	La medida real de la transformación
¿Puedo defenderla leyéndola?	Si se entiende o solo funciona
¿Cuánto costó encontrarla?	Si compensó frente a la vía directa
¿Sirve para otros problemas?	Si aprendiste algo o solo resolviste algo

💡 Detengámonos a pensar · La segunda pregunta es la importante

¿Qué pude borrar?

Es la medida que salió del capítulo 8, y la que conviene llevarse: si al terminar tienes todo lo que tenías más una estructura nueva, no has transformado el problema, lo has decorado.

En nuestro caso desaparecieron tres cosas: la lista de las ocho líneas, las coordenadas y la clasificación de centro, esquinas y lados. Esa es la medida.

Siete preguntas, entonces. Ninguna garantiza nada, y esa es la parte honesta del asunto.

Antes de que te lances a aplicarlas a todo lo que tengas encima, conviene el capítulo siguiente, que se ocupa de lo contrario: de cuándo este modelo no sirve, cuánto cuesta

descubrirlo tarde, y cómo reconocer que te has equivocado.

Capítulo 11

Cuándo esto no funciona

Un libro que solo cuenta cuándo su idea funciona no está enseñando una idea: está vendiéndola.

Este capítulo es el contrapeso de todo lo anterior. Va a decirte cuándo el modelo del capítulo 10 no sirve, cuándo aplicarlo sale caro y cómo darte cuenta de que te has equivocado.

💡 Detengámonos a pensar · Por qué esto no es humildad de cortesía

Hay una razón práctica, y es la que importa.

Si sales de este libro creyendo que todo problema esconde una transformación esperando, vas a perder mucho tiempo buscando en sitios donde no hay nada. Y después de perderlo tres veces, abandonarás el método entero.

Saber cuándo no aplicar algo es lo que permite seguir aplicándolo el resto de las veces.

11.1 Problemas que no admiten transformación útil

Empecemos por lo más incómodo: hay problemas cuya dificultad es genuina. No está en la descripción; está en el problema.

Cuando la dificultad es de tamaño, no de forma. Si hay que revisar un millón de expedientes porque son un millón y cada uno es distinto, ninguna representación va a hacer que sean menos. La forma no es el problema.

Cuando el problema ya está en su forma mínima. Ordenar una lista de números es ordenar una lista de números. Existen algoritmos mejores y peores, pero no hay una representación alternativa en la que ordenar deje de ser necesario.

Cuando lo esencial es irregular. Las transformaciones se apoyan en regularidades. Un problema cuyo contenido es precisamente la excepción —la legislación de un país, las reglas de un idioma, el historial de un paciente— no tiene una estructura oculta que lo ordene, porque su asunto *es* el desorden.

Cuando la dificultad es de juicio y no de cálculo. Decidir a quién contratar, qué priorizar o si un texto es bueno no se vuelve fácil cambiando la representación. Se puede representar mejor la información para decidir mejor, pero la decisión sigue siendo tuya.

Cómo distinguir los dos casos

Dificultad genuina

Dificultad genuina. La descripción del problema es tan compleja como el problema. Si te lo explican bien, entiendes por qué es difícil.

Ejemplo: predecir el tiempo a quince días.

Dificultad de representación

Dificultad de representación. La descripción es *desproporcionadamente* más compleja que el problema. Alguien te lo explica en un minuto y programarlo lleva cuatro mil líneas.

Ejemplo: el tres en raya.

La desproporción es la señal. Sin desproporción, probablemente no hay nada que buscar.

11.2 Cuando encontrarla cuesta más que resolver

Este es el límite más frecuente, y el que este libro tiene que mirar de frente porque le afecta directamente.

Encontrar el cuadrado mágico costó **varias horas** de una persona apartada del problema. Ese dato es del propio relato y no aparece en ninguna de las tablas del capítulo 14, porque no se puede medir con un cronómetro dentro de un programa.

Detengámonos a pensar · La cuenta honesta

Programar el tres en raya con condicionales y simetría: unas horas.

Encontrar la transformación: unas horas. Programarla después: menos de una.

Es decir: **en tiempo total, salieron parecidos**. La transformación no ganó por rapidez.

Ganó en lo que quedó al final: setenta y siete líneas que se pueden defender leyéndolas, frente a seiscientas treinta en las que hay que confiar.

Si lo único que necesitabas era un programa que funcionara, la vía directa era igual de buena. Es honesto decirlo.

Regla de bolsillo, imperfecta pero útil: **cuanto más veces vayas a usar la solución, y cuanto más tengas que responder por ella, más compensa el rodeo**. Para algo que se escribe una vez y se tira, no compensa casi nunca.

Inténtalo primero · Ponle presupuesto a la búsqueda

Antes de empezar a buscar una transformación, decide cuánto tiempo le vas a dar. En voz alta o por escrito.

Y decide también qué harás cuando se acabe: seguir por la vía directa.

Sin ese límite, buscar transformaciones se convierte en una forma elegante de no avanzar.

11.3 Cuando la dificultad solo se muda de sitio

Una transformación puede parecer un éxito y no haber eliminado nada: solo haber trasladado la complicación a otro rincón donde se ve menos.

Y no hace falta ir lejos para encontrar un ejemplo: está en este libro.

 Detengámonos a pensar · El ejemplo es nuestro propio caso

La formulación V2 del capítulo 8 —fichas 1 y 10 *más* cuadrado mágico para localizar la casilla— es exactamente esto.

Parecía que el cuadrado mágico aportaba algo. No aportaba nada: la línea amenazada tiene una sola casilla vacía, y localizarla por resta o mirarla directamente son la misma jugada. Comprobado: las mismas decisiones en las 723 posiciones comparadas.

V2 mantenía dos representaciones a la vez. Pagaba el coste de las dos y no podía descartar ninguna.

Señales de que la dificultad se mudó en lugar de desaparecer:

- Al terminar tienes **todo lo que tenías, más lo nuevo**.
- Hay que traducir constantemente de un lado a otro.
- Para explicar el programa hay que explicar las dos representaciones.
- No puedes borrar nada.

 Compruébalo · La prueba del borrado

Después de transformar, intenta **eliminar la representación original** del programa entero.

Si puedes, la transformación fue completa.

Si no puedes —si hay sitios donde sigue haciendo falta— has añadido una capa. Puede que aun así compense, pero llámalo por su nombre.

11.4 El riesgo de ver transformaciones donde no las hay

Hay un efecto secundario de aprender un método, y este libro puede provocarlo: empezar a ver su patrón en todas partes.

Coincidencias que no son correspondencias. Que dos cosas tengan el mismo número de elementos no significa que se correspondan. Ocho líneas y ocho tríos podrían haber sido casualidad; hizo falta comprobar que eran los *mismos* ocho, uno a uno. Sin esa comprobación, la correspondencia era una corazonada.

Analogías bonitas que no operan. Muchas analogías son buenas para explicar y no sirven para calcular. Si la analogía te ayuda a contar el problema pero no te permite deducir nada nuevo, es una metáfora, no una transformación.

Elegancia confundida con corrección. Una solución que sale redonda produce una satisfacción que se parece peligrosamente a la certeza. El capítulo 7 está en este libro precisamente por eso: las cuatro reglas eran elegantes y una de ellas estaba mal.

💡 Detengámonos a pensar · La prueba que separa una transformación de una metáfora

Una pregunta basta:

¿puedo deducir algo en la nueva representación que no supiera en la vieja, y traducirlo de vuelta?

Si sí, es una transformación.

Si solo sirve para contar el problema de otra manera, es una metáfora. Las metáforas son útiles para enseñar y no resuelven nada.

11.5 Cómo saber que te has equivocado

Termino con lo práctico: las señales de que conviene abandonar.

Cuadro 11.1: Señales de que la transformación no va a funcionar.

Lo que observas	Lo que probablemente significa
La traducción tiene excepciones	La correspondencia no es exacta
Necesitas las dos representaciones	Transformación parcial
No puedes borrar nada	Añadiste una capa
Explicarlo cuesta más que antes	Perdiste el beneficio principal
Llevas más tiempo buscando que resolviendo	Se acabó el presupuesto
Solo funciona en los casos que probaste	No era una propiedad, era suerte

💡 Detengámonos a pensar · Abandonar bien

Abandonar una transformación que no funciona no es fracasar. Es el resultado normal. Y no se vuelve con las manos vacías: has entendido el problema mejor que cuando empezaste, porque para buscar tuviste que abstraerlo, y esa abstracción te sirve igual por la vía directa.

Lo que sí es un fracaso es insistir en una transformación que ya diste por buena. Casi siempre porque era elegante.

Con esto queda dicho lo esencial sobre el modo de pensar que este libro propone: en qué consiste, de dónde salió, cómo se aplica y dónde se rompe.

Queda una cuestión que lo atraviesa todo y que hasta ahora solo ha ido asomando. Hoy no piensas solo: tienes al lado una herramienta que responde en segundos. La parte siguiente se ocupa de qué cambia eso, y de qué no cambia en absoluto.



Capacidades aumentadas con Inteligencia Artificial

Capítulo 12

La herramienta que amplifica

Este libro te ha pedido dos cosas contradictorias.

Te ha dado prompts para que construyas el algoritmo con tu asistente de inteligencia artificial, y te ha pedido que no le pidas la solución. Te ha dicho que uses la herramienta y que resistas la tentación de usarla.

Esta parte del libro explica por qué no es una contradicción.

12.1 Dónde estamos hoy

Conviene empezar por los hechos, porque sobre este asunto se habla mucho y se mide poco.

Cuadro 12.1: Adopción de la inteligencia artificial. Datos del informe anual del instituto *Human-Centered Artificial Intelligence* de la Universidad de Stanford (Stanford Institute for Human-Centered Artificial Intelligence, [2026](#)).

Indicador	Valor
Organizaciones que declaran usar IA (2025)	88 %
Estudiantes universitarios que usan IA generativa	4 de cada 5
Adopción entre la población, en tres años	53 %

Sea cual sea tu postura sobre todo esto, la discusión sobre si conviene usarla ha quedado atrás. Se usa. La discusión que queda, y que es la de este libro, es *cómo*.

💡 Detengámonos a pensar · Un dato que conviene tener presente

En el mismo informe aparece una brecha llamativa: el 73 % de los expertos espera un efecto positivo sobre el desempeño laboral, frente a solo el 23 % del público (Stanford Institute for Human-Centered Artificial Intelligence, [2026](#)).

No voy a decirte quién tiene razón. Sí quiero señalar que una diferencia de cincuenta puntos entre quienes conocen la herramienta y quienes la sufren o la temen es, en sí misma, un dato sobre lo mal explicada que está.

12.2 Lo que la IA hace extraordinariamente bien

Empecemos por reconocer la potencia, sin regatear.

En pruebas de resolución de problemas de programación reales, el rendimiento de los modelos pasó **del 60 % a casi el 100 % en un solo año** (Stanford Institute for Human-

Centered Artificial Intelligence, 2026). Eso incluye el tipo de tarea que este libro te ha pedido en cada etapa: escribe una función que haga esto, comprueba aquello, recorre este árbol.

Traducido a tu experiencia con este libro: si le pediste a tu asistente cualquiera de los prompts de la Parte III, casi con seguridad funcionó a la primera.

Cuadro 12.2: Dónde la herramienta rinde mejor.

Tarea	Por qué encaja
Escribir código a partir de una especificación	La especificación la pones tú
Explicar código ajeno	Hay un texto que interpretar
Traducir entre lenguajes o formatos	Correspondencia bien definida
Generar casos de prueba	Enumeración sistemática
Recorrer exhaustivamente algo pequeño	Trabajo mecánico, sin criterio
Resumir, reformular, corregir	Transformaciones sobre texto

Fíjate en lo que tienen en común: en todas ellas **el qué está decidido** y lo que falta es el cómo.

12.3 Lo que sigue siendo humano

Ahora la otra mitad, y aquí hay que andarse con cuidado porque es terreno de opiniones fuertes.

Empiezo por un hecho medido, no por una opinión. Los mismos modelos que alcanzan medalla de oro en la Olimpiada Internacional de Matemáticas leen correctamente la hora en un reloj de agujas **el 50,1 % de las veces** (Stanford Institute for Human-Centered Artificial Intelligence, 2026).

💡 **Detengámonos a pensar · Qué demuestra ese dato y qué no**

Demuestra que la competencia de estos sistemas es *desigual* de una manera que no se parece a la competencia humana. Nadie que resuelva problemas de olimpiada falla la mitad de los relojes.

No demuestra que las máquinas no piensen. Es una afirmación mucho más grande, es objeto de debate abierto, y este libro no está en condiciones de zanjarla.

La distinción importa. Un libro que usara ese dato para concluir lo segundo estaría haciendo exactamente lo que el capítulo 7 advertía: tomar una evidencia parcial por una demostración.

Dicho eso, aquí está la posición de este libro, declarada como posición:

Las máquinas emulan el pensamiento y el razonamiento de forma notablemente acertada, pero no los igualan.

Es discutible y la doy por discutible. Lo que sí puede afirmarse sin controversia es algo más modesto y bastante más útil, porque no depende de cómo se resuelva ese debate:

El reparto que sí es seguro

Lo que hace la herramienta

Lo que la herramienta hace. Producir. Escribir el código, explicarlo, corregirlo, probarlo, recorrer el árbol entero del juego en un segundo.

Todo eso lo hace mejor y más rápido que tú, y no tiene sentido competir.

Lo que no se delega

Lo que nadie hace por ti.

Notar que algo no encaja. La desproporción entre nueve casillas y cuatro mil quinientos casos no la señala ninguna máquina: la nota quien la mira y le parece excesiva.

Formular la pregunta. «¿Qué otra cosa tiene esta misma forma?» no se le ocurre a un asistente. Es la que tú le haces.

Decidir el criterio. Que la solución debía entenderse por completo, y no solo funcionar, fue una elección. Otra persona habría elegido la velocidad y habría llegado a otro sitio.

Responder por el resultado. Aunque una máquina razonara, no es ella quien da explicaciones cuando algo sale mal.

Detengámonos a pensar · El argumento que no depende del debate

Aunque mañana se demostrara que las máquinas razonan igual que nosotros, el reparto de la derecha seguiría en pie.

Porque no descansa en qué puede hacer la máquina, sino en **quién responde**. Y de eso respondes tú, con independencia de quién haya escrito el código.

12.4 Poner en marcha habilidades que no dominas

Aquí está la posibilidad genuinamente nueva, y es la razón por la que este libro trae prompts.

Hasta hace poco, para seguir un razonamiento como el de la Parte III había que saber programar. No porque el razonamiento fuera técnico —no lo es: son sumas y cuatro reglas— sino porque comprobarlo exigía un oficio.

Eso ha cambiado. Hoy puedes pedirle a tu asistente que construya cada pieza mientras tú haces lo verdaderamente difícil: entender por qué esa pieza es la adecuada, y comprobar que hace lo que dice.

Sin programar · Lo que has hecho realmente si no programas

Si has seguido el libro con los prompts, no has «hecho trampa». Has hecho una cosa distinta y perfectamente legítima:

Has **dirigido** una construcción. Has decidido qué había que construir y en qué orden, has sabido qué esperar de cada pieza y has comprobado si llegaba. Esa es la parte que no se delega.

Lo que has delegado es la escritura. Es exactamente lo que un arquitecto delega en un albañil sin dejar de ser el arquitecto.

💡 Detengámonos a pensar · La condición que hace que esto funcione

Apoyarte en habilidades que no dominas solo funciona si te apoyas **desde las que sí dominas**.

Si no sabes programar pero entiendes el problema, puedes dirigir la construcción. Si no entiendes el problema, no estás dirigiendo nada: estás pidiendo y aceptando.

La diferencia entre las dos cosas es toda la diferencia, y no depende de saber programar.

12.5 El riesgo: una herramienta que desalienta la pausa

Y ahora el problema, que es real y este libro no puede esquivar.

Todo lo que la Parte II defendía —detenerse, apartarse, dedicar horas a no resolver— se apoya en soportar un rato de desconcierto. Ese rato es donde aparece la pregunta que replantea el problema.

Una herramienta que responde en segundos **elimina ese rato**.

⚖️ Los dos caminos, desde el mismo punto de partida

Pedir la solución

Escribes: «hazme un tres en raya que no pierda nunca».

En un minuto tienes código que funciona. Probablemente con Minimax, probablemente correcto.

Has resuelto el problema y no has aprendido nada. Ni siquiera te has enterado de que había otra manera de mirarlo, porque nunca llegaste a mirarlo.

Pensar primero

Escribes las instrucciones a mano. Cuentas los casos. Te incomoda la desproporción. Te apartas. Vuelves con otra pregunta.

Tardas una tarde en llegar donde el otro camino llegaba en un minuto.

Pero ahora sabes por qué funciona, puedes defenderlo, y tienes una pregunta —«¿qué otra cosa tiene esta forma?»— que te va a servir el resto de tu vida profesional.

💡 Detengámonos a pensar · Por qué insisto en esto

Ya quedó dicho en el capítulo 3, y lo repito aquí a propósito porque allí era una advertencia y ahora es una conclusión: nunca fue tan fácil no detenerse.

Lo que entonces solo podía enunciarse, ahora puedes contrastarlo con tu propia experiencia leyendo este libro. Si usaste los prompts sin intentar los retos, tienes un algoritmo y

no tienes una pregunta. Si los intentaste, sabes de qué rato estoy hablando.

Esto no es un argumento en contra de la herramienta. Sería absurdo: este libro, por el contrario, invita a utilizarla paso a paso, especialmente a quienes no son programadores.

Es un argumento sobre el **orden**: primero la pregunta, después la herramienta. Y existe una regla sencilla para saber si estamos respetando ese orden:

✓ Compruébalo · Antes de escribir un prompt

Pregúntate si sabes **qué le estás pidiendo y por qué eso y no otra cosa**.

Si lo sabes, la herramienta te está amplificando.

Si no lo sabes, la herramienta está decidiendo por ti, y lo que salga no podrás defenderlo.

Primero la pregunta, después la herramienta. Es fácil de enunciar y difícil de sostener cuando la herramienta está a un teclazo de distancia.

El capítulo siguiente lo baja a lo concreto: cómo se escribe una petición que te obligue a pensar en lugar de ahorrarte el pensamiento. Los prompts de este libro están contruidos así, y conviene saber por qué.

Capítulo 13

Prompts que hacen pensar

Los prompts de este libro están escritos de una manera muy concreta, y no por casualidad. Este capítulo explica el criterio, para que puedas escribir los tuyos.

No hay aquí trucos de redacción ni fórmulas mágicas. Hay una sola idea, y es la misma del libro entero aplicada a otra cosa.

13.1 No pidas la solución, pide acompañamiento

Dos maneras de pedir lo mismo

Pedir el resultado

«Escríbeme un programa que juegue al tres en raya sin perder, y que además detecte las jugadas dobles.»

Recibes código que funciona. No sabes por qué funciona, no elegiste el enfoque, y si mañana falla no sabrás dónde mirar. La herramienta ha resuelto *su* versión del problema, que es la que ella supone que querías.

Pedir acompañamiento

«Necesito representar el estado de la partida así: cada jugador posee un conjunto de números del 1 al 9...Escríbeme el código que represente ese estado...Explicame en dos frases por qué esta representación no necesita coordenadas.»

Recibes una pieza que tú decidiste, en el sitio donde la querías, con una explicación que puedes contrastar. Es, literalmente, el prompt de la etapa 1.

El enfoque lo has elegido tú. La herramienta ha construido.

La diferencia no está en el resultado inmediato: los dos funcionan. Está en quién decidió, y por tanto en quién puede responder por lo que salió.

Detengámonos a pensar · La regla de una línea

Pide piezas de un plan que tú tienes. No pidas el plan.

Si no tienes plan, el problema no es el prompt: es que todavía no has pensado el problema. Vuelve a la Parte II.

13.2 Anatomía de un buen prompt

Mira uno de los prompts de la Parte III y verás siempre las mismas cinco piezas.

Cuadro 13.1: Las cinco piezas de un prompt que acompaña. Los ejemplos son literales de los prompts de las etapas 1 y 2 del capítulo 6.

Pieza	Qué hace	Ejemplo del libro
Contexto	Sitúa el trabajo	«Vamos a construir juntos, paso a paso...»
Límite	Impide adelantarse	«No te adelantes a las siguientes»
Encargo	Dice qué construir	«Escribe una función que reciba...»
Restricción	Fija tu decisión	«No uses ninguna lista de líneas...»
Devolución	Te obliga a pensar	«Explícame en dos frases por qué...»

Las dos últimas son las que casi nadie escribe, y son las importantes.

La restricción es donde vive tu criterio. «No uses coordenadas» no es un capricho: es la decisión de diseño que hace que el algoritmo sea el que queremos. Sin ella, el asistente usaría una cuadrícula, porque es lo más frecuente, y el libro entero se habría evaporado en un prompt.

La devolución convierte la respuesta en una comprobación. Si le pides además que explique por qué algo sobra, y su explicación no te convence, has detectado un problema que el código funcionando te habría ocultado.

💡 Detengámonos a pensar · Por qué la devolución funciona

Cuando pides una explicación junto al código, obtienes dos cosas que puedes contrastar entre sí.

Si el código hace una cosa y la explicación dice otra, hay un error, y lo has encontrado sin ejecutar nada.

Es la misma idea que verificar un algoritmo por dos caminos distintos. Aquí los dos caminos son el código y su relato.

13.3 Los prompts de este libro, explicados

Vale la pena mirar tres de ellos y ver qué hace cada pieza. De cada uno digo dónde está entero, para que puedas leerlo completo y comprobar que la pieza es la que aquí se le atribuye.

13.3.1 El de la etapa 2

Está en el capítulo 6, recuadro «Etapa 2: detectar la victoria».

*«En el cuadrado mágico, las ocho líneas ganadoras del tres en raya son exactamente los ocho conjuntos de tres números del 1 al 9 que suman 15. Escribe una función que... **No uses ninguna lista de líneas ni coordenadas del tablero.** Después dime cuántas líneas has tenido que escribir explícitamente en el código, y compáralo con las que necesitarías si trabajaras sobre la cuadrícula.»*

La restricción impide la solución habitual. La devolución te hace medir la diferencia en lugar de crearla. Ninguna de las dos pide más código: piden *atención*.

13.3.2 El de la comprobación

Está en el capítulo 7, recuadro «Comprobar que nunca pierde».

«Recorre el árbol COMPLETO del juego. . . El objetivo es cero derrotas. Si aparece aunque sea una, muéstrame la partida que lleva a ella e identifica qué regla falló.»

Este prompt está escrito **contando con el fallo**. No pide una confirmación: pide una investigación, y le dice qué hacer si el resultado es malo.

Y menos mal, porque el resultado *fue* malo: así descubrí que la cuarta regla estaba mal especificada.

💡 **Detengámonos a pensar · Nunca escribas un prompt que solo pueda confirmarte**

Un prompt que dice «comprueba que mi algoritmo funciona» es una invitación a que te den la razón.

Uno que dice «encuentra el caso donde falle, y si no lo encuentras dime cuántos casos has revisado» es una invitación a mirar.

La diferencia es la misma que entre buscar pruebas de que tienes razón y buscar la verdad.

13.3.3 El de la búsqueda de estructuras

Está en el capítulo 10, recuadro «Buscar estructuras con la misma forma».

*«Te voy a describir un problema **sin decirte de qué va**, a propósito. . . No quiero que lo resuelvas. Quiero que me digas qué otras estructuras matemáticas, juegos, o problemas conocidos tienen esta misma forma. . . No descartes ninguno por raro. Los raros son los que me interesan.»*

Este es el más raro de los tres y el más interesante. Prohíbe explícitamente resolver. Está usando la herramienta para el movimiento 3 del modelo, que es el que no se puede sistematizar.

Ocultar el dominio no es un juego: es lo que impide que el asistente responda con lo que ya se hace en ese campo, que es justo lo que no queremos.

13.4 Cómo pedirle a la IA que te lleve la contraria

Un asistente tiende a ser complaciente. Le presentas una idea y encontrará razones para que sea buena. No es maldad ni adulación: es que le has dado un punto de partida y trabaja desde él.

La solución es pedirle explícitamente lo contrario.

Prompt · Que refute tu transformación

Te voy a proponer una transformación de un problema, y quiero que intentes demostrarme que NO funciona.

Mi problema es: [describelo].

Mi transformación propuesta es: [describela].

No busques razones para que funcione. Busca:

1. Un caso donde la correspondencia falle: algo que exista en el problema original y no tenga equivalente en el transformado, o al revés.
2. Algo que se conserve en el original y se pierda al traducir.
3. Una solución que sea válida en el dominio transformado pero que al traducirla de vuelta no signifique nada.

Si después de buscarlo no encuentras ningún fallo, dime exactamente qué has comprobado y qué no has podido comprobar. No me digas que está bien: dime qué has mirado.

Detengámonos a pensar · Por qué esto vale más que un elogio

Una transformación que resiste tres intentos de refutación es más sólida que una que ha recibido tres aprobaciones.

Y hay un beneficio secundario: los intentos de refutación fallidos te enseñan dónde está la fuerza de tu idea, que es algo que la aprobación nunca te dice.

13.5 Verificar lo que te responde

Última pieza, y la que no se puede delegar.

Todo lo que un asistente te devuelve es una **propuesta**, incluidas las explicaciones. Puede estar bien, puede estar sutilmente mal, y lo segundo es más peligroso porque parece lo primero.

Cuadro 13.2: Qué verificar y cómo, sin necesidad de programar.

Si te devuelve...	Comprueba...
Un número	Recalcúlalo por otra vía
Una lista	Que no falte ni sobre ninguno
Una afirmación general	Un caso concreto donde debería cumplirse
Código	Que la explicación diga lo que el código hace
Una cita o una fuente	Que exista, y que diga lo que se le atribuye
Un «funciona»	Cuántos casos ha revisado para decirlo

✓ Compruébalo · La verificación que este libro usó siempre

Cuando el problema es pequeño, no hay que confiar en nada: se recorre entero. Los ocho tríos que suman quince no me los creí: enumeré los ochenta y cuatro subconjuntos posibles. Las cero derrotas no las estimé: jugué todas las partidas. Antes de aceptar cualquier resultado, pregúntate si tu problema se puede recorrer completo en un ordenador. Se puede más veces de las que parece.

💡 Detengámonos a pensar · La última responsabilidad

Puedes delegar la escritura, el cálculo, la exploración y buena parte de la comprobación. No puedes delegar la aceptación. Cuando das algo por bueno, lo estás firmando tú. Ese es el motivo real por el que este libro insiste tanto en entender la solución además de tenerla: no se puede firmar lo que no se entiende.

Y aquí termina lo que este libro tenía que enseñarte.

Queda el balance: reunir todas las cifras que han ido apareciendo, decir cómo se obtuvieron y separar con cuidado lo que demuestran de lo que no. Va al final a propósito, para que puedas juzgarlo con el algoritmo ya construido y comprobado por ti.

VI

El balance

Capítulo 14

Análisis de resultados

A lo largo del libro han ido apareciendo cifras. Este capítulo las reúne, dice cómo se obtuvieron y —lo que más importa— separa lo que demuestran de lo que no.

Va aquí, al final, por una razón deliberada. Si estuviera al principio, parecería la justificación de una tesis decidida de antemano. Puesto al final, después de que hayas construido el algoritmo y comprobado tú mismo lo que podías comprobar, es lo que debe ser: un balance.

14.1 Qué medí y cómo

Toda cifra de este libro está **contada o medida sobre código ejecutado**. Ninguna es una estimación, un orden de magnitud ni un recuerdo.

Comparé tres soluciones al mismo problema, todas correctas y todas verificadas:

Cuadro 14.1: Las tres soluciones comparadas.

	Cómo decide	Qué necesita saber
A	Un caso por posición	Todas las posiciones del juego
B	Un caso por clase de simetría	Íd., agrupadas por equivalencia
C	Cuatro reglas aritméticas	Que las líneas suman quince

A y B no las escribí a mano —sería inviable—: las generé automáticamente a partir de la solución exacta del juego, de modo que son las mejores versiones posibles de sí mismas. C es la que hemos construido juntos.

💡 Detengámonos a pensar · Por qué esto importa más de lo que parece

Es tentador comparar una solución cuidada propia contra una versión descuidada de la ajena. El resultado sale siempre favorable y no significa nada.

Generar A y B automáticamente elimina esa tentación: no hay margen para que estén mal escritas.

14.2 Número de casos que cubre cada solución

Primero, el tamaño real del problema. El capítulo 2 dio las cifras y prometió que aquí estaría la cuenta. Aquí está.

Empieza por lo más grosero. Cada casilla está vacía, con tu ficha o con la del rival: $3^9 = 19\,683$ combinaciones. Pero casi ninguna puede ocurrir en una partida, porque los turnos se alternan y nadie juega dos veces seguidas.

Impón esa alternancia. Sea k el número de casillas ocupadas, con $0 \leq k \leq 9$. Quien empezó puso $\lceil k/2 \rceil$ fichas y el otro las $\lfloor k/2 \rfloor$ restantes. Elegir qué casillas están ocupadas y cuáles de ellas son de quien empezó da

$$\sum_{k=0}^9 \binom{9}{k} \binom{k}{\lceil k/2 \rceil} = 6\,046 \quad (14.1)$$

tableros con recuentos de fichas legales. Siguen sobrando: en 568 de ellos alguien tenía ya tres en línea antes de la última jugada, de modo que la partida habría terminado y ese tablero no se alcanza nunca. Descontados, quedan 5 478 posiciones alcanzables.

De esas 5 478, en 958 no hay nada que elegir: alguien ganó o no queda casilla libre. Las 4 520 restantes son exactamente las que un programa tiene que cubrir.

Cuadro 14.2: El universo del tres en raya. Cada paso se calcula a partir del anterior, salvo las cifras marcadas con *: esas exigen recorrer el juego, no se deducen de una fórmula.

Magnitud	Cómo se obtiene	Valor
Combinaciones de las nueve casillas	3^9	19 683
Recuentos de fichas legales	ecuación 14.1	6 046
Exigían una victoria anterior	recorrido*	568
Posiciones alcanzables	$6\,046 - 568$	5 478
Ya decididas, sin jugada que elegir	recorrido*	958
Hay que decidir jugada, empezando	recorrido*	2 423
Hay que decidir jugada, respondiendo	recorrido*	2 097
Casos que cubrir	$5\,478 - 958 = 2\,423 + 2\,097$	4 520
Posiciones legales alcanzables	$2 \times 5\,478 - 2\,423$	8 533
Íd. módulo las ocho simetrías	recorrido*	1 192

La penúltima fila merece una palabra. Las 5 478 son las posiciones que aparecen si abres tú. Si abre el rival aparecen otras tantas, y no son las mismas: las 2 423 que comparten los dos papeles son justamente aquellas en las que te toca mover habiendo empezado tú. De ahí 8 533 posiciones distintas para un programa que deba jugar en ambos papeles.

✓ **Compruébalo · El cálculo, término a término**

La suma de la ecuación 14.1 cabe en una tabla y se hace a mano. $\binom{9}{k}$ elige qué casillas están ocupadas; $\binom{k}{\lceil k/2 \rceil}$ reparte esas k fichas entre quien abrió y quien respondió.

k	$\binom{9}{k}$	$\binom{k}{\lceil k/2 \rceil}$	producto
0	1	1	1
1	9	1	9
2	36	2	72
3	84	3	252
4	126	6	756
5	126	10	1 260
6	84	20	1 680
7	36	35	1 260
8	9	70	630
9	1	126	126
total			6 046

Lo que la fórmula no da —las 568 inalcanzables, las 958 ya decididas, el reparto 2 423/2 097 y las 1 192 clases de simetría— sale de recorrer el juego entero. Es la parte de este recuento que hay que ejecutar en lugar de calcular, y por eso el capítulo 7 insiste tanto en ella.

Y ahora, cuántas situaciones distintas debe cubrir cada solución para decidir una jugada:

Cuadro 14.3: Casos que debe cubrir cada solución.

	Empezando	Respondiendo	Total
A) Enumeración exhaustiva	2 423	2 097	4 520
B) Con simetría	338	289	627
C) Cuadrado mágico	—	—	4 reglas

La casilla vacía de C no es un descuido. C no enumera casos: decide con cuatro reglas, sea cual sea la posición. No hay número que poner ahí.

14.3 Tamaño del código

Cuadro 14.4: Tamaño de cada implementación, medido.

	Líneas de código	Condicionales	Bytes
A) Enumeración exhaustiva	4 522	4 520	239 435
B) Con simetría	630	627	35 215
C) Cuadrado mágico	77	14	5 116

Cuadro 14.5: Reducción obtenida por la transformación.

Reducción de C	frente a A	frente a B
Líneas de código	59×	8,2×
Bytes	47×	6,9×
Condicionales	323×	44,8×

 Detengámonos a pensar · Una salvedad honesta

Las líneas y los bytes de A y B dependen del estilo con que se genera la tabla. Otro generador daría otras cifras, aunque del mismo orden.

La magnitud robusta, la que no depende de cómo se escriba, es el **número de casos** —4520 y 627 frente a cuatro reglas— y el número de condicionales.

Si alguna vez te enseñan una comparación de este tipo sin esta salvedad, desconfía.

14.4 Coste por jugada

El tamaño del código no lo es todo. Conviene ver también qué cuesta decidir una jugada.

Cuadro 14.6: Coste de decidir una jugada.

	Operaciones por jugada	Memoria
A) Cadena de condicionales	$\approx 2\,260$ comparaciones	4 520 casos
A) Con tabla asociativa	1 búsqueda	4 520 entradas
B) Cadena con simetría	≈ 313 comparaciones	627 casos
C) Cuadrado mágico	52,5 de media	8 rectas

En C, las reglas 1 y 2 cuestan **24 operaciones fijas**: dieciséis sumas para las ocho rectas y ocho comparaciones. El máximo observado, 464 operaciones, corresponde a las posiciones donde se dispara la regla 3, que sí simula respuestas del rival.

 Dónde reside la corrección

Pensamiento tradicional

En la tabla. El programa es correcto porque los casos están bien.

Si te preguntan por qué juega bien, la respuesta honesta es: «porque lo hemos probado».

Transformación estructural

En una propiedad matemática y cuatro reglas.

Si te preguntan por qué juega bien, puedes explicarlo. Y quien te escuche puede comprobarlo.

14.5 Verificación: cero derrotas

La condición del encargo era no perder nunca. Lo verifiqué recorriendo el árbol completo del juego —todas las respuestas del rival, en las dos posiciones de salida y explorando también las alternativas del propio azar—, no una muestra.

Cuadro 14.7: Necesidad de las reglas 3 y 4, por ablación.

Configuración	Partidas	Derrotas
Reglas 1, 2, 4 aleatoria	86 048	8 320
Reglas 1, 2, 3, 4 aleatoria	39 232	1 600
Reglas 1, 2 + 4 posicional	2 096	64
Reglas 1, 2, 3 + 4 posicional	1 784	0

Degradar la regla 4 a azar puro, o quitar la regla 3, produce derrotas. El orden de las reglas 1 y 2 se comprueba aparte, en el capítulo 7: invertirlas hace que el algoritmo defienda una amenaza teniendo una victoria a mano. Las cuatro son necesarias, y juntas son suficientes.

Verifiqué también las tres formulaciones del capítulo 8:

Cuadro 14.8: Las tres formulaciones, verificadas.

Formulación	Partidas	Derrotas
V1 — fichas 1/10, sin cuadrado mágico	551	0
V2 — fichas 1/10 con cuadrado mágico	551	0
V3 — solo aritmética, sin geometría	551	0

Además, V1 y V2 toman la misma decisión en las 723 posiciones comparadas.

14.6 La comparación justa: simetría en ambos lados

Hay una trampa fácil en este tipo de comparaciones, y conviene desactivarla.

La simetría se presenta habitualmente como la mejora del enfoque por condicionales. Pero **el algoritmo del cuadrado mágico también mejora con ella**. Reducir el tablero a su forma canónica no cambia ninguna decisión, y en cambio reduce el número de posiciones distintas que hay que considerar y verificar: de 361 a 116 clases.

Usar la simetría solo en un lado de la comparación inflaría el resultado a favor de C. La comparación de este capítulo la aplica a ambos.

💡 Detengámonos a pensar · Una regla general para leer comparaciones

Cuando veas dos soluciones comparadas, pregúntate si las mejoras disponibles se han aplicado a las dos.

Casi nunca se han aplicado. Y casi nunca es de mala fe: es que uno cuida más lo suyo.

14.7 Qué demuestran estas cifras

Con toda la cautela puesta, esto es lo que las mediciones establecen:

La detección de amenazas se reduce a sumar. Saber qué líneas están ganables o amenazadas cuesta veinticuatro operaciones fijas, frente a recorrer una estructura o consultar una tabla de miles de entradas.

Las reglas pasan de miles a cuatro. No es una reducción de estilo: son 4 520 casos frente a cuatro reglas, y 627 frente a cuatro cuando se aplica la mejor optimización conocida del enfoque tradicional.

La corrección se vuelve defendible leyendo. En A y B hay que confiar en la tabla. En C, cuatro reglas y una propiedad demostrable.

Una transformación completa rinde; una parcial, no. V2 mantiene las dos representaciones a la vez y no obtiene nada del cuadrado mágico. V3 lo lleva hasta el final y elimina la geometría entera.

14.8 Qué no demuestran

Y esto es lo que no establecen, por mucho que las cifras impresionen:

No demuestran que la transformación resuelva el juego. Dos de las cuatro reglas —evitar dobles amenazas y la preferencia posicional— son política de juego, no consecuencia del cuadrado mágico. La transformación abarata su comprobación; no las suministra.

No demuestran que esto funcione en otros problemas. Un caso es un caso. El tres en raya tenía una estructura matemática esperando, y no hay razón para suponer que todos los problemas la tengan.

No miden lo que costó encontrar la transformación. Fueron varias horas de una persona. Ese coste no aparece en ninguna tabla y es real: si el problema se resuelve en diez minutos por la vía directa, no compensa.

No dicen nada sobre problemas grandes. Todo lo verificado aquí se apoya en que el tres en raya se puede recorrer completo. En cuanto un problema no se puede recorrer, la verificación exhaustiva deja de estar disponible y hay que volver a confiar.

💡 Detengámonos a pensar · Por qué este apartado existe

Un libro que solo enseñara la tabla anterior sería más persuasivo y menos útil.

La costumbre de decir qué no demuestra lo que uno acaba de demostrar es parte del método que este libro intenta transmitir. Si el capítulo terminara en la sección anterior, estaría enseñando lo contrario de lo que predica.

14.9 Cómo reproducir las mediciones

Nada de esto hay que creérselo. El tres en raya se puede recorrer completo en un ordenador cualquiera, y por eso todas las cifras de este capítulo son reproducibles en unos minutos.

 Prompt · Reproducir la comparación completa

Quiero reproducir una comparación entre tres formas de resolver el tres en raya. Hazlo paso a paso y muéstrame los resultados en tablas.

1. Cuenta cuántas posiciones legales alcanzables tiene el juego, y cuántas quedan si agrupas las que son equivalentes por rotación o reflexión.
2. Cuenta en cuántas de esas posiciones le toca decidir a un jugador. Ese es el número de casos que necesitaría una solución escrita como una lista de condicionales.
3. Genera automáticamente esa lista de condicionales a partir de la solución exacta del juego, y mide cuántas líneas y cuántos bytes ocupa. Repítelo agrupando por simetría.
4. Verifica recorriendo el árbol completo del juego, en las dos posiciones de salida, y dime cuántas derrotas tiene cada una.

Al final, dime en qué se parecen y en qué se diferencian tus cifras de estas: 8533 posiciones, 4520 casos, 627 con simetría, 0 derrotas.

 Compruébalo · Si tus cifras no coinciden

Las diferencias pequeñas suelen venir de definir «posición alcanzable» de otro modo: si cuentas también las posiciones ya terminadas, o si permites posiciones ilegales con fichas de más.

Lo que debe coincidir sin margen es lo importante: los ocho tríos que suman quince, y las cero derrotas.

Y con el balance hecho, el libro termina aquí.

Lo que queda establecido es modesto de enunciar y no de consecuencias: en este problema, cambiar la representación convirtió miles de casos en cuatro reglas, y la corrección pasó de residir en una tabla a poder defenderse leyendo. Lo que no queda establecido está dicho con la misma claridad dos secciones más arriba.

Lo que sigue va anexo: un glosario, unas lecturas y un cierre.

Apéndice A

Glosario

Los términos aparecen aquí con el significado que tienen *en este libro*, que no siempre es el único posible. Cuando un término tiene un uso más amplio fuera de aquí, se indica.

Términos del método

Pensar fuera de la caja En este libro, no significa «ser creativo» ni «tener ideas raras». Significa algo concreto: *poner en duda la descripción heredada del problema antes de intentar resolverlo*. La caja es la descripción, no la falta de imaginación.

Problema Lo que hay que conseguir: los estados posibles, las reglas que los relacionan y la condición que decide cuándo se ha logrado. Es lo que no cambia aunque cambiemos de representación.

Representación La forma concreta en que escribimos un problema para poder manipularlo. Un mismo problema admite muchas. Todas describen lo mismo; no todas hacen visible lo mismo.

Transformación El paso de una representación a otra conservando lo que decide si el problema está resuelto. Para ser válida debe cumplir tres cosas: correspondencia de estados, conservación de la condición de éxito y posibilidad de volver.

Transformación completa Aquella tras la cual la representación original puede descartarse por entero.

Transformación parcial Aquella en la que la representación nueva convive con la vieja sin sustituirla. Se paga su coste y no se cobra su beneficio. Ver capítulo 8.

Invariante Algo que se mantiene igual pase lo que pase. En una transformación, lo que hay que conservar para que la solución siga siendo válida. En una partida, lo que hace legal una secuencia de jugadas: un movimiento por turno, la misma ficha por jugador, alternancia respetada.

Incubación Nombre que recibe en psicología el efecto de apartarse deliberadamente de un problema. Estudiado, discutido y no concluyente. En este libro, «soltar la papa caliente».

Desproporción La señal que conviene aprender a reconocer: cuando la descripción de un problema resulta mucho más compleja que el problema. No demuestra que haya una transformación esperando, pero es donde vale la pena buscar.

Términos matemáticos

Cuadrado mágico Matriz cuadrada que contiene cada entero de 1 a n^2 exactamente una vez, y en la que cada fila, cada columna y cada diagonal principal suman lo mismo. De orden tres solo hay uno, salvo simetrías.

Constante mágica El valor común de esas sumas. Para orden tres, quince. Se deduce, no se memoriza: $K_n = n(n^2 + 1)/2$.

Biyección Correspondencia uno a uno entre dos conjuntos, sin sobrantes por ninguno de los dos lados. Es lo que garantiza que al traducir no se pierda ni se invente nada.

Isomorfismo Correspondencia que además conserva la estructura, y no solo los elementos. Dos juegos isomorfos son el mismo juego con distinto disfraz.

Grupo diédrico D_4 Las ocho transformaciones que dejan igual a un cuadrado: cuatro rotaciones y cuatro reflexiones. Actúa tanto sobre el tablero como sobre el cuadrado mágico.

Clase de equivalencia Conjunto de elementos que, a efectos del problema, cuentan como el mismo. Las cuatro esquinas del tablero forman una.

Forma canónica El representante elegido de una clase de equivalencia. Reducir a forma canónica no cambia ninguna decisión y reduce el número de situaciones distintas que hay que considerar.

Codificación posicional Asignar valores de modo que la posición de cada cifra en el resultado signifique algo. Con fichas 1 y 10, la suma de una línea vale $a + 10b$: las unidades cuentan tus fichas y las decenas las del rival.

Términos del juego

Línea ganadora Cualquiera de los ocho conjuntos de tres casillas que dan la victoria: tres filas, tres columnas, dos diagonales. En la representación aritmética, cualquiera de los ocho tríos que suman quince.

Amenaza Situación en la que un jugador posee dos casillas de una línea y la tercera está libre. Se detecta porque la línea vale 2 o 20.

Doble amenaza Dos amenazas simultáneas. Solo puede bloquearse una, de modo que la partida está perdida. Evitar que se produzca es la tercera regla del algoritmo.

Complemento El número que falta para completar una línea: $15 - (a + b)$.

Casilla muerta Aquella cuya línea ya no puede completarse porque el rival ocupa alguna de sus casillas.

Términos de verificación

Verificación exhaustiva Comprobar *todos* los casos posibles, no una muestra. Solo está disponible cuando el problema se puede recorrer completo. El tres en raya lo permite; casi ningún problema real lo permite.

Ablación Quitar una parte del algoritmo para ver si su ausencia produce fallos. Es como se comprobó que las cuatro reglas son necesarias y no solo suficientes.

Defensa por lectura Poder explicar por qué un programa funciona *siempre* leyéndolo, y no solo por qué hizo una cosa concreta. Es distinto de la trazabilidad, y es lo que este libro persigue.

Términos de inteligencia artificial

Prompt La instrucción que se le da a un asistente. En este libro, nunca una petición de solución: siempre el encargo de una pieza dentro de un plan que tienes tú.

Restricción La parte del prompt donde vive tu criterio. «No uses coordenadas» no es un capricho: es la decisión de diseño que hace que el resultado sea el que buscabas.

Devolución La parte del prompt que te obliga a pensar: pedir además una explicación que puedas contrastar con el código. Si ambos no dicen lo mismo, has encontrado un error sin ejecutar nada.

Minimax Algoritmo clásico para juegos por turnos: explora las jugadas posibles y las respuestas del rival, y elige el camino que lleve al mejor desenlace suponiendo que el rival también juega bien. Es la solución habitual al tres en raya, y la que yo no conocía.

Apéndice B

Para seguir leyendo

Esta lista es corta a propósito. Cada entrada dice por qué está aquí y qué esperar de ella, para que puedas decidir si te interesa antes de buscarla.

Todas las referencias se han comprobado: existen, dicen lo que aquí se les atribuye, y sus datos están verificados. Es lo mínimo que se le puede pedir a un libro que ha dedicado un capítulo entero a insistir en verificar.

B.1 Sobre el caso de este libro

El origen documentado de la correspondencia

Michon, J. A. (1967). *The game of JAM: An isomorph of tic-tac-toe*. The American Journal of Psychology, 80(1), 137–140 (Michon, [1967](#)).

Es la fuente primaria de lo que este libro construye desde cero. Michon describe formalmente que el juego *JAM* —elegir números del uno al nueve y ganar reuniendo tres que sumen quince— es el tres en raya con otro disfraz, y que el puente entre ambos es el cuadrado mágico de orden tres.

Léelo si quieres ver la misma equivalencia enunciada desde la psicología experimental en lugar de desde la programación. Y léelo, sobre todo, para comprobar por tu cuenta lo que el capítulo 5 reconoce: la correspondencia estaba descrita mucho antes de que este libro existiera.

El juego aparece también con los nombres *Number Scrabble*, *Pick 15* y *3 to 15*, por si buscas por ahí.

B.2 Sobre cómo se resuelven problemas

El clásico que sigue vigente

Pólya, G. (2004). *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, colección Princeton Science Library. Publicado originalmente en 1945 (Pólya, [2004](#)).

Ochenta años después sigue siendo el mejor libro sobre el asunto, y no por nostalgia: porque se ocupa de lo que casi nadie se ocupa, que es qué preguntas hacerse cuando uno no sabe cómo seguir.

El modelo de siete movimientos del capítulo 10 le debe bastante, aunque Pólya no habla de cambiar la representación sino de atacar el problema desde donde se pueda. Su pregunta más útil, y la más parecida al espíritu de este libro, es: «¿conoces algún problema relacionado?».

Hay traducción al español con el título *Cómo plantear y resolver problemas*.

💡 **Detengámonos a pensar · Una advertencia sobre esta lectura**

Pólya escribe para quien resuelve problemas de matemáticas, con enunciado cerrado y solución conocida por alguien.

Buena parte de sus consejos se trasladan bien a problemas abiertos, pero no todos. Léelo sabiendo eso y te ahorrarás forzar analogías.

B.3 Sobre el estado de la inteligencia artificial

Datos en lugar de opiniones

Stanford Institute for Human-Centered Artificial Intelligence (2026). *The 2026 AI Index Report* (Stanford Institute for Human-Centered Artificial Intelligence, 2026).

Es de donde salen las cifras del capítulo 12. Se publica cada año, es gratuito y está lleno de gráficas.

Lo recomiendo por un motivo concreto: sobre inteligencia artificial se escriben muchas afirmaciones rotundas y se miden pocas. Este informe mide. No te dirá qué pensar, y precisamente por eso es útil.

Busca en él las secciones donde el rendimiento es desigual —extraordinario en unas tareas, sorprendentemente flojo en otras—, que es donde más se aprende.

B.4 Sobre apartarse del problema

Lo que se sabe y lo que no

Kuliev, N. S. (2024). *Incubation processes in solving problems: Current state and prospects of research*. *Journal of Modern Foreign Psychology*, 13(4), 131–139 (Kuliev, 2024).

Una revisión reciente de lo que la investigación dice sobre la incubación, es decir, sobre apartarse deliberadamente de un problema.

La incluyo con una advertencia que ya aparece en el capítulo 3 y que conviene repetir: **esta revisión no concluye que apartarse funcione**. Señala que las definiciones no están estandarizadas, que los diseños experimentales varían mucho y que los resultados no son concluyentes.

La recomiendo justamente por eso. Si vas a apoyar tu forma de trabajar en la pausa —como hace este libro— conviene que sepas que la evidencia es discutida, y no fiarte de quien te diga lo contrario. Incluido yo.

B.5 Sobre reformular un problema

Lo que la ciencia cognitiva llama reestructuración

Wiley, J., y Danek, A. H. (2023). *Restructuring processes and Aha! experiences in insight problem solving*. *Nature Reviews Psychology*, 3, 42–55 (Wiley & Danek, 2023).

Si un solo artículo se acerca al asunto de este libro desde la investigación académica, es este. Lo que aquí se ha llamado «cambiar la representación», en psicología cognitiva se llama *reestructuración*: el momento en que quien resuelve reinterpreta el problema y la solución aparece.

Es una revisión, de modo que sirve para ver el mapa completo del campo sin tener que entrar en cada estudio.

💡 Detengámonos a pensar · Lo que este artículo no dice

Conviene leerlo sabiendo esto, porque es fácil sacar la conclusión que uno ya llevaba puesta.

La revisión recoge **dos posturas** en tensión: quienes identifican la comprensión súbita con la reestructuración y quienes la identifican con la experiencia de «¡ajá!». Y observa que *no siempre coinciden*: se reestructura sin sentir nada, y se siente el «¡ajá!» sin que haya habido reestructuración.

Es decir: reformular un problema y tener una revelación son cosas distintas, aunque a menudo se cuentan como si fueran la misma. Este libro se ocupa de la primera.

B.6 Sobre enseñar a pensar así

Qué se hace hoy en las aulas

Wu, T.-T., Asmara, A., Huang, Y.-M., y Permata Hapsari, I. (2024). *Identification of problem-solving techniques in computational thinking studies: Systematic literature review*. *SAGE Open*, 14(2) (Wu et al., 2024).

Revisión de treinta y siete estudios sobre cómo se enseña el llamado *pensamiento computacional*. Es útil para situar lo que este libro propone dentro de lo que ya se hace.

Sus cuatro etapas más empleadas son descomposición, reconocimiento de patrones, abstracción y algoritmo. Fíjate en cuál **no** está: cambiar la representación del problema. La abstracción se le acerca —quitar lo no esencial es el segundo movimiento del capítulo 10— pero el paso de buscar otra estructura con la misma forma no aparece como etapa propia.

No lo señalo como carencia del campo, que sabe más que este libro. Lo señalo porque explica por qué el movimiento 3 le resulta extraño a casi todo el mundo: no se enseña.

Referencias

- Kuliev, N. S. (2024). Incubation processes in solving problems: Current state and prospects of research. *Journal of Modern Foreign Psychology*, 13(4), 131-139. <https://doi.org/10.17759/jmfp.2024130412>
- Michon, J. A. (1967). The game of JAM: An isomorph of tic-tac-toe. *The American Journal of Psychology*, 80(1), 137-140. <https://doi.org/10.2307/1420555>
- Pólya, G. (2004). *How to Solve It: A New Aspect of Mathematical Method* (Princeton Science Library) [Publicado originalmente en 1945]. Princeton University Press.
- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson.
- Sergeyuk, A., Titov, S., Golubev, Y., & Bryksin, T. (2023). Overcoming the mental set effect in programming problem solving. *Proceedings of the 34th Annual Workshop of the Psychology of Programming Interest Group (PPIG'23)*. <https://doi.org/10.48550/arXiv.2307.06673>
- Stanford Institute for Human-Centered Artificial Intelligence. (2026). *The 2026 AI Index Report*. Stanford University. Consultado el 22 de agosto de 2026, desde <https://hai.stanford.edu/ai-index/2026-ai-index-report>
- Wang, S., Okada, T., & Takagi, K. (2023). How to effectively overcome fixation: A systematic review of fixation and defixation studies on the basis of fixation source and problem type. *Frontiers in Education*, 8, 1183025. <https://doi.org/10.3389/feduc.2023.1183025>
- Wiley, J., & Danek, A. H. (2023). Restructuring processes and Aha! experiences in insight problem solving. *Nature Reviews Psychology*, 3, 42-55. <https://doi.org/10.1038/s44159-023-00257-x>
- Wu, T.-T., Asmara, A., Huang, Y.-M., & Permata Hapsari, I. (2024). Identification of problem-solving techniques in computational thinking studies: Systematic literature review. *SAGE Open*, 14(2). <https://doi.org/10.1177/21582440241249897>

Más allá de la caja

Cambiar la forma de representar para cambiar la forma de resolver

Si hubiera que reducir este libro a una frase, sería esta: **el problema no siempre está donde lo estamos mirando.**

El tres en raya no era difícil. Difícil era la descripción que arrastrábamos de él: nueve casillas, ocho líneas, miles de casos. Cuando escribí el mismo juego con los números de un cuadrado mágico, la mayor parte de esa dificultad no se resolvió: dejó de existir. No había estado nunca en el juego. Estaba en cómo lo escribíamos.

Eso es lo que aquí se ha llamado pensar fuera de la caja, y conviene decirlo sin adornos. No consiste en resolver mejor. Consiste en *detenerse antes de resolver*, el tiempo suficiente para preguntar si la forma en que tenemos planteado el problema es la que nos conviene. Es una pausa breve y contraria a todos nuestros hábitos: lo aprendido, lo premiado y lo evaluado es empezar rápido.

Quien haya seguido el libro construyendo el algoritmo lo habrá comprobado en primera persona. La pregunta que abrió la solución no fue «¿cómo resuelvo esto?», sino «¿qué otra cosa tiene esta misma forma?». La primera encierra en el problema. La segunda obliga a salir de él.

La IA como ampliación de lo humano

Toda herramienta que el ser humano ha construido amplía algo que ya tenía. La palanca no inventó la fuerza: la multiplicó. La escritura no inventó la memoria: la extendió más allá de una vida. El ábaco, la imprenta y el ordenador siguieron el mismo patrón. Ninguna sustituyó a la capacidad que amplificaba; todas la llevaron más lejos de lo que su dueño podía llevarla solo.

La inteligencia artificial ocupa hoy ese lugar. Con una diferencia de grado que conviene reconocer sin dramatismo: por primera vez la herramienta amplifica capacidades que considerábamos exclusivamente nuestras.

El capítulo 12 se ocupó de ese reparto con detalle. Aquí basta recordar cuál era el fondo del asunto, porque no depende de ningún debate sobre si las máquinas piensan.

Puedes delegar la escritura, el cálculo, la exploración y buena parte de la comprobación. Lo que no se delega es **notar que algo no encaja, formular la pregunta, decidir el criterio y responder por el resultado**. Aunque una máquina razonara, no es ella quien da explicaciones cuando algo sale mal.

Y de ahí venía la invitación de este libro, y el motivo de que cada etapa llegara acompañada de su prompt: si no programas, has podido dirigir una construcción sin escribirla; si programas, se te ha pedido lo contrario, que resistas la tentación de pedir la solución hecha.

Queda una frase, que es la que resume el asunto entero:

La inteligencia artificial multiplica lo que llevas puesto. Si llevas preguntas, multiplica preguntas. Si llevas prisa, multiplica prisa.

El valor de la pregunta

Queda una última cosa por decir, y es la más difícil de enseñar.

Todo este libro nació de una molestia. Miré un programa de miles de condicionales para un juego de nueve casillas y pensé que aquello era desproporcionado. Podría no haberlo pensado. El programa funcionaba.

Esa molestia no es un método, no se puede sistematizar del todo y no hay prompt que la produzca. Pero se puede cultivar, y se cultiva de una sola manera: dándose permiso para dudar de lo que ya funciona.

Si este libro te deja algo, ojalá sea esa incomodidad productiva. La próxima vez que resuelvas un problema y la solución te parezca más complicada que el problema, detente. Puede que no haya nada. Pero puede que estés a una pregunta de distancia de verlo todo de otra manera.

La próxima entrega: Space Invaders y la orientación a objetos

Este libro se ha ocupado de un problema pequeño y de una sola idea: cambiar la representación antes de resolver. El tres en raya servía porque puedes recorrerlo completo, incluso de cabeza, y porque permite comprobar cada afirmación sin creer a nadie.

La próxima entrega cambia de escenario y de escala: **exploraremos la programación orientada a objetos utilizando Space Invaders.**

La elección no es caprichosa. El tres en raya es un problema *estático*: un estado, una decisión, un turno. Space Invaders introduce todo lo que aquí faltaba, y que la orientación a objetos existe precisamente para gobernar: muchas entidades a la vez, cada una con su estado y su comportamiento; interacción entre ellas; tiempo que transcurre aunque nadie juegue; y una frontera clara entre lo que el juego *es* y cómo se ve en pantalla.

Habrà, como aquí, retos antes de las respuestas, prompts para quien prefiera construirlo con su asistente, y propuestas para llevarlo al aula. Y habrá, sobre todo, la misma pregunta de fondo aplicada a un problema donde las respuestas son bastante menos evidentes: *¿estamos mirando este problema desde el sitio correcto?*